

“Design and Development of a Hotel Check-In and Check-Out Portal”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

By

BAGGU LAKSHMI SAHITHI 2520040111

CHILUPURI SHRADDHA 2520040105

KATHULA TANIYA 2520040083

Under the Esteemed Guidance of

Dr Karumuri Sri Rama Murthy

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING



Declaration

The Project Report entitled **“Design and Development of a Hotel Check-In and Check-Out Portal”** is a record of Bonafide work of **BAGGU LAKSHMI SAHITHI - 2520040111, CHILUPURI SHRADDHA - 2520040105, KATHULA TANIYA - 2520040083** submitted in partial fulfillment for the award of B. Tech in Electronics & Communication Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

BAGGU LAKSHMI SAHITHI - 2520040111

CHILUPURI SHRADDHA - 2520040105

KATHULA TANIYA - 2520040083

K L (Deemed to be) University

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **“Design and Development of a Hotel Check-In and Check-Out Portal”** is a bonafide work done and submitted by **BAGGU LAKSHMI SAHITHI - 2520040111, CHILUPURI SHRADDHA - 2520040105, KATHULA TANIYA - 2520040083**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Electronics & Communication Engineering, K L (Deemed to be University), during the academic year **2025-2026**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our guides **Dr. K Sree Rama Murthy**, **Dr. K Anusha**, Department of Computer Science which cannot be expressed in words for his and her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. S. Srinivasa Rao (ECE)**, Head of the **Department for Electronics & Communication Engineering** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

BAGGU LAKSHMI SAHITHI - 2520040111

CHILUPURI SHRADDHA - 2520040105

KATHULA TANIYA - 2520040083

Abstract

The hotel industry still faces delays and inconvenience during traditional front-desk check-in and check-out processes, especially during peak hours. Guests often spend unnecessary time waiting for room allocation, submitting identity documents, and completing payment formalities. Manual handling also increases the risk of errors in guest records, room assignment, and invoice generation. To solve this, a “**Hotel Check-In & Check-Out Portal**” is proposed as a web-based self-service system that streamlines guest entry and exit workflows digitally.

The portal allows guests to complete **online check-in** by filling out personal details, uploading ID proof, and receiving an automatically assigned room based on availability. A **digital key mock system** will simulate key access, enhancing the modern hotel experience. During check-out, guests can use an **express checkout feature** that calculates charges and generates a downloadable **invoice PDF** instantly.

The system also includes guest profile management, notifications for booking updates, stay history tracking, and an admin panel to manage rooms, guest records, and check-in/check-out requests.

This project will be developed using **React.js** for the frontend, **Node.js with Express.js** for backend APIs, and **MySQL (optional)** for database storage. It improves hotel service efficiency, reduces front-desk workload, and provides guests with a faster, smoother experience.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-10
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	11-15
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	16-25
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	26-30
5	Design	5.2 ER Diagram / Database schema	31
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	32-39
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	40-42
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	43-53
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	54-59
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	60-65
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	66-69
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	70-74
13	References	- Books, tutorials, APIs, documentation sites used	75-76

14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	77-80
----	------------	---	-------

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

The hospitality industry is a cornerstone of the global economy, and the efficiency of its core operations—particularly guest check-in and check-out—directly impacts customer satisfaction and business profitability. Traditionally, hotel front-desk operations have been heavily reliant on manual, paper-based processes. Guests arriving at a hotel often face long queues, repetitive form-filling, and delays in room allocation. This manual approach is not only time-consuming but also prone to errors in guest records, room assignment, and billing.

In recent years, the advent of digital technologies has transformed various industries, and the hospitality sector is no exception. The rise of contactless services, driven in part by health and safety concerns and the growing expectation for instantaneous service, has created a demand for self-service solutions. Guests now prefer the convenience of managing their stay details from their smartphones or laptops, minimizing physical interaction at the front desk.

This project, "Hotel Check-In & Check-Out Portal," emerges from the need to modernize these outdated processes. It aims to create a web-based self-service platform that empowers guests to manage their entire stay digitally. From pre-registering their details to checking out and generating invoices, the system is designed to be a one-stop solution for both guests and hotel staff.

By leveraging modern web technologies, the portal automates key tasks such as room assignment, guest registration, and billing. For hotel administrators, it provides a centralized dashboard to monitor occupancy, manage guest records, and streamline operations. This project directly addresses the inefficiencies of manual processes, aiming to reduce front-desk workload, minimize human error, and significantly enhance the overall guest experience by providing a faster, smoother, and more modern hotel stay.

1.2 Problem Statement

In today's fast-paced world, travellers expect efficiency and convenience at every touchpoint of their journey. However, the hotel industry often lags in this regard, with many establishments still relying on slow, manual, and error-prone front-desk operations. The core problem addressed by this project is the lack of a modern, integrated digital solution to streamline the guest check-in and check-out processes.

The **core problem** is the inefficiency and frustration caused by traditional hotel check-in and check-out processes, which are characterized by:

- **Long Reception Queues:** Guests face extended waiting times at the front desk due to manual check-in and identity verification processes.
- **Paper-Based Registration:** Manual, error-prone paperwork slows down room allocation and increases administrative overhead.
- **Slow Check-Out & Billing:** Manual invoice preparation causes delays at check-out, leading to frustration for both guests and staff.
- **Poor Record Management:** Difficulty maintaining accurate guest records and booking history hampers reporting and operational insights.

This outdated approach significantly detracts from the guest experience, especially during peak hours. It places an unnecessary burden on hotel staff, leading to inefficiency and increased operational costs. Furthermore, it creates a barrier to offering a truly modern and "contactless" guest journey.

The project, "Hotel Check-In & Check-Out Portal," aims to solve these problems by developing a self-service web portal that automates the guest journey. The solution will:

1. **Eliminate Waiting Times** by allowing guests to check-in online before arrival or upon arrival via a kiosk or mobile device.
2. **Digitize Registration** with a user-friendly online form that auto-fills data and integrates with a room inventory system.
3. **Automate Billing and Check-out** by instantly generating accurate invoices and allowing express check-out.
4. **Improve Record Management** by providing a centralized, digital database for all guest information and stay history.

By digitizing these core operations, the portal aims to significantly reduce front-desk workload, minimize manual errors, enhance operational efficiency, and deliver a superior, frictionless experience for the modern traveller.

1.3 Scope of the Project

The scope of this project, "Hotel Check-In & Check-Out Portal," encompasses the design, development, and implementation of a comprehensive, web-based digital hotel management system. The project will focus on creating a Minimum Viable Product (MVP) that digitizes the core guest journey from check-in to check-out, providing a robust solution for small to medium-sized hotels or as a standalone module for larger systems.

The **technical scope** involves building a fully responsive, web-based application. The system will be developed using React.js for the frontend, Node.js with Express.js for the backend APIs, and will use LocalStorage for data persistence in the initial MVP. This architecture ensures a modern, efficient, and scalable application.

The **functional scope** includes the following key modules:

1. **User Authentication:** Guest signup, login, and logout with secure password validation.
2. **Digital Check-In:** An online form to capture guest details (name, ID, contact) with automatic room assignment from a predefined inventory.
3. **Room & Digital Key Display:** A dashboard for guests to view their assigned room number and a simulated digital key feature.
4. **Express Check-Out:** A one-click checkout feature that records departure time, archives the guest data, and triggers invoice generation.
5. **Invoice Generation:** Generation of a downloadable PDF invoice containing guest details, stay dates, and room charges.
6. **Stay History:** A section for guests to view their past and current stays with status badges (Active/Checked Out) and search functionality.
7. **Admin Dashboard:** A dashboard for hotel staff providing real-time statistics (total guests, occupied/available rooms, today's check-ins, recent guests).
8. **Guest Profile:** A personal profile section for users to manage their information.

The **out of scope** for this MVP includes actual payment gateway integration, integration with IoT devices (like real digital locks), a separate mobile application, multi-property management, integration with third-party booking platforms (like [Booking.com](https://www.booking.com)), detailed staff role management, housekeeping modules, and real-time SMS/email notifications.

The **geographical scope** is global, as the web-based nature of the portal allows it to be used by any hotel property regardless of location.

1.4 Objectives of the Project

The main objective of this project is to design and develop a self-service digital hotel management portal that streamlines the guest check-in and check-out processes, reducing front-desk workload and enhancing the guest experience through automation and intelligent design.

The **primary objectives** are:

1. **To Digitize Operations:** Replace paper-based registration and manual billing with a seamless digital workflow.
2. **To Reduce Wait Time:** Eliminate long queues at reception through online check-in and express check-out features.
3. **To Empower Admins:** Provide hotel staff with a powerful dashboard to manage guests, rooms, and operational records.

The **technical objectives** are:

1. To develop a responsive and user-friendly web application using React.js.
2. To build a robust backend API with Node.js and Express.js for handling business logic and data management.
3. To ensure secure user authentication using JWT (JSON Web Tokens).
4. To implement a persistent data storage solution, initially using LocalStorage.
5. To create a modular and maintainable codebase that follows industry best practices.

The **user-centered objectives** are:

1. To allow guests to complete check-in in under 3 minutes.
2. To eliminate manual errors in room assignment.
3. To provide instant PDF invoices at check-out.
4. To enable guests to view their complete stay history.
5. To provide a self-service solution that reduces the need for front-desk interaction.
6. To ensure a responsive, intuitive, and accessible user experience across all devices.

The **business objectives** are:

1. To reduce front-desk workload by 60%.
2. To improve staff efficiency by automating administrative tasks.
3. To provide management with real-time insights into occupancy and guest activity.
4. To enhance the hotel's brand image by offering a modern, contactless guest experience.

In summary, the project aims to bridge the gap between traditional hotel operations and the expectations of the modern, digital-savvy traveller, creating a win-win solution for both guests and hotel management.

1.5 Importance of the Application

The importance of the "Hotel Check-In & Check-Out Portal" application is rooted in its potential to fundamentally transform the guest experience and operational efficiency of the hospitality industry. In a world where speed, convenience, and personalization are paramount, this application provides a crucial digital solution that addresses the shortcomings of traditional hotel operations.

For Guests:

The portal significantly enhances the guest journey by offering unparalleled convenience and speed. It eliminates the frustration of waiting in long queues after a tiring journey. Guests can complete the entire check-in and check-out process from their personal devices, at their own pace. This self-service approach not only saves time but also provides a sense of control. Features like instant invoice generation and a digital record of their stay add transparency and value, contributing to a more positive and memorable experience.

For Hotel Staff and Management:

The application automates repetitive and time-consuming administrative tasks. By shifting routine activities like data entry and billing to the digital portal, front-desk staff are freed to focus on more meaningful guest interactions and high-value tasks. This directly reduces the risk of human error in room assignment and billing, improving accuracy and trust. The admin dashboard provides critical real-time insights into occupancy, guest flow, and operational performance, enabling data-driven decision-making for staffing, pricing, and resource allocation.

For the Business:

The implementation of such a system leads to direct operational efficiencies and cost savings. By reducing the time spent per guest, hotels can serve more guests with the same staff, improving throughput and profitability. The modern, tech-savvy image projected by such a portal can be a key differentiator in a competitive market, attracting a new generation of travelers who value digital-first experiences. Furthermore, the ability to generate detailed guest history and stay records facilitates personalized marketing and better customer relationship management (CRM).

From a **technological and academic perspective**, the project demonstrates the practical application of modern full-stack web development. It showcases the integration of a responsive frontend (React.js), a scalable backend (Node.js/Express.js), and a persistent data layer (LocalStorage/Database). This makes it a valuable case study for software engineering principles, including API design, state management, user authentication, and UI/UX design.

In conclusion, the application is important not just as a piece of software, but as a catalyst for a more efficient, guest-centric, and technologically advanced hospitality industry. It represents a significant step towards the future of hotel management, where technology serves as the backbone of a superior and seamless guest experience.

1.6 Target Users / Audience

The target users of the Hotel Check-In & Check-Out Portal are diverse, encompassing the primary stakeholders in the hotel ecosystem who will interact with and benefit from the system.

Primary Users: Guests

This is the largest group of users. They are the customers who will use the portal for self-service check-in, digital key access, express check-out, and viewing their stay history. This group includes:

- **Individual Travelers:** Business professionals, solo adventurers, and leisure travellers who value speed and convenience.
- **Families and Groups:** Who can manage multiple bookings from a single interface.
- **Tech-Savvy Travelers:** Who prefer digital, contactless, and self-service options over traditional front-desk interactions.
- **Returning Guests:** Who will value the system's ability to remember their profile and stay history, making repeat bookings even faster.

Secondary Users: Hotel Front Desk Staff

These are hotel employees who are the primary beneficiaries of the system's automation. The portal reduces their manual workload, allowing them to focus on more complex guest requests and service recovery. They use the admin dashboard to monitor occupancy, manage guest records, and oversee daily operations. Their role shifts from data entry to guest service and operational oversight.

Tertiary Users: Hotel Administrators and Management

This group includes property managers, general managers, and owners. They use the system for high-level oversight and strategic decision-making. The portal provides them with valuable data through the admin dashboard, such as occupancy trends, average stay durations, and revenue per room. This data enables them to make informed decisions on pricing, promotions, and resource planning.

Potential Future Users:

- **Developers and System Integrators:** Who may extend the application or integrate it with other hotel systems (PMS, Booking Engines).
- **Hotel Chains:** As the system can be adapted for multi-property management in future iterations.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The hardware requirements for the "Hotel Check-In & Check-Out Portal" project are designed to support efficient development and smooth deployment while ensuring accessibility for end-users. The requirements are categorized into two sections: development environment and end-user/client access.

Development Environment:

The development team requires a robust hardware setup to handle the multiple tasks of coding, testing, and running the application. The recommended hardware specifications are:

- **Processor:** Intel Core i5 (6th Generation or higher) or AMD Ryzen 5 (or higher). A multi-core processor is recommended for handling various development tools, virtual machines, and heavy browser tabs.
- **Memory (RAM):** A minimum of 8 GB of RAM is essential, while 16 GB is highly recommended. This allows for smooth operation of the IDE (e.g., Visual Studio Code), Node.js server, and multiple Chrome browser windows for testing.
- **Storage:** An SSD (Solid State Drive) of 256 GB or more is preferred for faster data read/write speeds, improving overall system performance and application load times.
- **Operating System:** The development OS can be Windows 10/11, Ubuntu Linux, or macOS, based on developer preference. Linux is often favored for its compatibility with Node.js environments and open-source tools.
- **Internet Connection:** A stable broadband connection with a minimum speed of 10 Mbps is necessary for installing dependencies, integrating with GitHub, and fetching any external API data.
- **Monitor:** A Full HD (1920x1080) monitor is recommended for UI/UX development to view the application at a reasonable scale.
- **Web Browser:** Google Chrome, Mozilla Firefox, or Microsoft Edge (latest versions) are required for testing the application's frontend.

End-User/Client Hardware:

The hardware requirements for guests and hotel staff using the portal are minimal. Since the application is web-based, users simply need a device with a modern web browser and an internet connection.

- **Device:** Desktop computer, laptop, tablet, or smartphone.
- **Processor:** Any modern processor (dual-core or higher).
- **Memory:** A minimum of 4 GB of RAM is sufficient for running a web browser smoothly.
- **Storage:** No significant storage is required on the client side.
- **Browser:** Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge (latest versions).
- **Screen Resolution:** The system's responsive design ensures it functions correctly on screen sizes ranging from 5 inches (smartphones) to large monitors.

2.2 Software Requirements

The software requirements define the digital ecosystem and technologies necessary for building, testing, and deploying the Hotel Check-In & Check-Out Portal. The selection of tools and frameworks is based on their suitability for building a modern, scalable, and maintainable web application.

Operating System:

- **Development:** Windows 10 or 11, Ubuntu 22.04 LTS, or macOS. Ubuntu is preferred for its open-source nature and seamless integration with web development tools.
- **Server (if not using a PaaS):** Linux (Ubuntu 22.04 LTS) is recommended for its stability, security, and performance.
- **Client:** The OS is irrelevant as the application is accessed via a web browser.

Server-Side Technologies (Backend):

- **Runtime Environment:** Node.js (version 18.x LTS or higher), chosen for its asynchronous, event-driven architecture, which is excellent for handling multiple concurrent API requests.
- **Web Framework:** Express.js, a lightweight and flexible Node.js framework, for building RESTful APIs and managing routing, middleware, and server-side logic.
- **Database (Optional for Future Enhancements):** MySQL or MongoDB for persistent data storage. A hybrid approach with MySQL for structured data (bookings, payments) and MongoDB for unstructured data (profiles, history) is a potential future upgrade. For the MVP, data will be persisted using `LocalStorage`.
- **Authentication:** JSON Web Tokens (JWT) for stateless user authentication and authorization. Bcrypt.js for hashing user passwords securely.
- **Security:** Helmet.js for securing HTTP headers, CORS for managing cross-origin requests, and express-validator for input sanitization.

Client-Side Technologies (Frontend):

- **Core Framework:** React.js (v18.x), for building a dynamic, responsive, and component-based user interface.
- **State Management:** React Context API or Redux Toolkit for managing global application state, such as user login status and notification messages.
- **Styling:** Bootstrap 5 for a responsive grid system and pre-built UI components, along with custom CSS for unique branding and styling.
- **Routing:** React Router (v6.x) for implementing client-side routing and creating a Single Page Application (SPA).
- **HTTP Client:** Axios for making asynchronous HTTP requests from the frontend to the backend API.
- **PDF Generation:** jsPDF (or similar) for generating downloadable PDF invoices at check-out.

Development and Collaboration Tools:

- **Code Editor:** Visual Studio Code (VS Code) with extensions for React, JavaScript, and Prettier/ESLint.
- **Version Control:** Git, hosted on GitHub, for source code management, collaboration, and change tracking.
- **API Testing:** Postman for testing and validating REST API endpoints.
- **Package Managers:** npm (Node Package Manager) or Yarn for managing project dependencies.

2.3 Development Tools and Frameworks

The development of the Hotel Check-In & Check-Out Portal is powered by a carefully selected set of modern tools and frameworks. These technologies are the foundation of the project's software engineering process, ensuring efficiency, scalability, and high quality across all phases of development—from coding and testing to deployment.

Frontend Technologies (UI & UX):

- **React.js (v18.x):** This is the core frontend library. It allows for the creation of a dynamic and highly responsive user interface by breaking it down into reusable components. Its virtual DOM ensures fast rendering, which is critical for a smooth, app-like user experience.
- **React Router (v6.x):** Used to implement client-side routing, enabling navigation between pages (e.g., Home, Check-In, Dashboard, History) without refreshing the entire page, creating a fluid Single Page Application (SPA) experience.
- **Bootstrap 5:** A popular CSS framework used to ensure a responsive and mobile-first design. It provides a grid system and pre-built components like navbars, forms, modals, and cards, which expedite development and ensure visual consistency.
- **React Context API / Redux Toolkit:** For state management, these tools help manage global data (like the user's authentication status, current guest details, and notification states) across the entire application, ensuring consistency and easy data sharing between components.
- **Axios:** A promise-based HTTP client for the browser and Node.js, used to make asynchronous API calls to the backend, handling request/response data effortlessly.
- **jsPDF:** A library for generating PDF files on the client side, used to create the downloadable invoice at the time of guest check-out.

Backend Technologies (Server & Logic):

- **Node.js (v18.x LTS):** The server-side JavaScript runtime. Its event-driven, non-blocking I/O model is perfect for building scalable network applications, making it ideal for handling the API requests generated by the frontend.
- **Express.js:** A minimal and flexible Node.js web application framework that provides a robust set of features for building web and mobile applications. It is used to create the RESTful APIs that handle all server-side logic.
- **JSON Web Tokens (JWT):** An open standard for creating access tokens. JWT is used for stateless authentication, securely transmitting user information between the client and server.
- **Bcrypt.js:** A library to help hash passwords, adding a critical layer of security to user authentication.

Database Technologies (Data Persistence):

- **LocalStorage:** For the MVP, the browser's LocalStorage API is used for data persistence. This allows for a fully functional application without a separate database server, ideal for prototyping and demonstration.

- **MySQL / MongoDB (Planned for Future):** These are considered for future iterations to handle larger datasets and multi-user scenarios. MySQL is a strong choice for relational data (bookings, users), while MongoDB offers flexibility for unstructured data (reviews, logs).

Testing and Quality Assurance:

- **Postman:** The primary tool for testing and validating API endpoints.
- **Selenium WebDriver:** Used for automated frontend UI testing.
- **Jest / React Testing Library:** For unit and integration testing of React components.
- **Git & GitHub:** The industry standard for version control and collaborative development.

Deployment and Hosting:

- **Vercel / Netlify:** For hosting the frontend React application.
- **Heroku / AWS EC2:** For deploying the backend Node.js API.
- **GitHub Actions:** For creating a Continuous Integration and Continuous Deployment (CI/CD) pipeline.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The front-end of the Hotel Check-In & Check-Out Portal is the critical interface through which users (guests and administrators) interact with the system. Its design philosophy is centered on creating a seamless, intuitive, and responsive experience that simplifies the hotel management process. Built using React.js and Bootstrap 5, the frontend prioritizes usability, accessibility, and a professional aesthetic.

Design Principles and UX:

The core purpose of the frontend is to guide users effortlessly through the entire guest journey. The design emphasizes a clean, uncluttered layout with a clear visual hierarchy. The color scheme and typography are chosen to convey professionalism and trust, making the experience welcoming for guests. Key UX principles include:

- **Simplicity:** The interface minimizes cognitive load by presenting only essential information at each step.
- **Clarity:** Actions and navigation paths are explicit, ensuring users are never confused about what to do next.
- **Consistency:** A uniform design language is applied across all pages, fostering familiarity and ease of use.
- **Feedback:** The system provides immediate and clear feedback for all user actions (e.g., success/error notifications).

Architecture and Structure:

The frontend is built using a component-based architecture in React.js. This means the UI is broken down into small, reusable pieces, such as `<NavBar />`, `<CheckInForm />`, `<RoomCard />`, and `<AdminDashboard />`. This modularity not only makes the code easier to maintain but also ensures consistency across the application.

- **HTML5:** Provides the semantic structure for all web pages.
- **CSS3 & Bootstrap 5:** Responsible for styling and responsive layout. Bootstrap's grid system is heavily utilized to ensure the portal adapts flawlessly to different screen sizes (desktops, tablets, and smartphones). Custom CSS is used for specific brand elements and unique design touches.
- **JavaScript (ES6+):** Drives all interactivity, from dynamic form validation to rendering real-time data on the admin dashboard.

Functionality and Interactivity:

- **Dynamic Forms:** The check-in form automatically populates user data from their profile and uses real-time validation to ensure all information is accurate before submission.
- **Data Visualization:** The admin dashboard uses simple cards and charts to display key metrics like occupancy and revenue, making data easy to digest at a glance.

- **Notifications:** In-app notifications provide real-time feedback for user actions (e.g., "Check-in successful," "Your invoice is ready").
- **Responsive Navigation:** A responsive navbar collapses on smaller screens, ensuring easy access to all features from any device.

Performance Optimization:

- **Code Splitting & Lazy Loading:** React features like `React.lazy` and `Suspense` are used to load only the necessary components for a given page, reducing initial load times.
- **Image Optimization:** All images are compressed and served in modern formats (WebP) where possible to minimize download sizes.
- **Caching:** Browser caching is leveraged to store static assets, resulting in faster load times for returning users.

3.2 Back-End Design

The back-end of the Hotel Check-In & Check-Out Portal serves as the powerful engine behind the user interface, responsible for all server-side logic, data processing, and secure communication. Designed for speed, reliability, and security, it uses Node.js with the Express.js framework to build a robust RESTful API that connects the frontend to the data layer.

Core Architecture:

The back-end follows a Model-View-Controller (MVC)-like architecture.

- **Model:** Represents the data structure. For the MVP, data is managed via utility functions that interact directly with `LocalStorage`, but the code is structured to easily swap in a database ORM like `Sequelize` (for SQL) or `Mongoose` (for MongoDB) in the future.
- **View:** In this context, the "view" is the JSON response sent back to the client (frontend).
- **Controller:** Contains the business logic for each API endpoint. For example, a `checkInController.js` file would handle the logic for validating input, assigning a room, and saving the guest data.

API Design and Communication:

The platform uses RESTful APIs for communication. All requests from the React frontend are sent over HTTPS to specific endpoints.

- **Endpoint Structure:** Routes are logically grouped (e.g., `/api/auth/login`, `/api/guest/checkin`, `/api/admin/dashboard`, `/api/guest/checkout`).
- **Data Format:** All data is exchanged using JSON (JavaScript Object Notation), ensuring lightweight and standardized communication.
- **HTTP Methods:** The API correctly uses GET (for fetching data), POST (for creating new records), PUT (for updating records), and DELETE (for removing records).

Security Implementation:

Security is a top priority for the backend.

- **Authentication:** Users are authenticated using JSON Web Tokens (JWT). When a user logs in, a secure JWT token is generated and signed with a secret key. This token must be sent in the `Authorization` header of every subsequent API request.
- **Authorization:** Middleware functions verify the JWT token before granting access to protected routes. This ensures that only authenticated users can perform actions like checking in or viewing their history.
- **Password Hashing:** User passwords are never stored in plain text. The `bcryptjs` library is used to hash passwords before they are saved.
- **Input Sanitization & Validation:** `express-validator` is used as middleware to validate and sanitize all incoming user data, protecting the application against common threats like SQL injection and XSS (Cross-Site Scripting).

- **Middleware:** The application uses middleware for various purposes, including Helmet.js for securing HTTP headers and CORS for managing cross-origin requests.
- **Error Handling:** A global error-handling middleware catches exceptions and returns consistent, user-friendly error messages with appropriate HTTP status codes.

Data Management and Persistence:

- **LocalStorage:** For the MVP, the back-end server's API will interact with the browser's LocalStorage via the frontend. This provides a simple and complete solution for development and demonstration.
- **Future Database:** The API is designed to be decoupled from the data source, making it easy to connect to a real database (like MySQL or MongoDB) for production.

3.3 Database Design

The database design for the Hotel Check-In & Check-Out Portal is a critical component, serving as the central repository for all guest information, booking details, and operational data. The design is focused on ensuring data integrity, efficiency, and scalability. For the MVP, the system will use LocalStorage for simplicity and portability, but a formal database schema has been designed for future implementation with a relational database like MySQL.

The system's data is organized into several key entities. The primary entities include **User**, **Room**, **GuestStay**, and **Booking**. The relationships between these entities form the core of the application's data logic.

Core Entities and Data Model:

1. **User:** This table stores information about registered guests and hotel staff.
 - `user_id` (Primary Key, INT AUTO_INCREMENT)
 - `name` (VARCHAR 255)
 - `email` (VARCHAR 255, UNIQUE)
 - `password_hash` (VARCHAR 255, hashed using bcrypt)
 - `role` (ENUM 'guest', 'staff', 'admin')
 - `phone` (VARCHAR 20)
 - `id_proof_type` (VARCHAR 50)
 - `id_proof_number` (VARCHAR 100)
 - `created_at` (TIMESTAMP)
2. **Room:** This table manages the hotel's inventory of rooms.
 - `room_id` (Primary Key, INT AUTO_INCREMENT)
 - `room_number` (INT, UNIQUE)
 - `room_type` (VARCHAR 50, e.g., 'Standard', 'Deluxe', 'Suite')
 - `floor` (INT)
 - `capacity` (INT)
 - `price_per_night` (DECIMAL 10,2)
 - `status` (ENUM 'available', 'occupied', 'cleaning', 'under_maintenance')
 - `description` (TEXT)
3. **GuestStay:** This table records the actual check-in and check-out events, linking users to rooms.
 - `stay_id` (Primary Key, INT AUTO_INCREMENT)
 - `guest_id` (Foreign Key referencing User.user_id)

- `room_id` (Foreign Key referencing Room.room_id)
- `check_in_date` (DATETIME)
- `check_out_date` (DATETIME, can be NULL for active stays)
- `status` (ENUM 'active', 'checked_out', 'cancelled')
- `total_charges` (DECIMAL 10,2)
- `invoice_number` (VARCHAR 50, UNIQUE)

Future Enhancements:

For future scalability and to handle a larger volume of transactions, the schema can be extended to include other tables such as `Payment`, `Booking`, and `HousekeepingTask`.

3.4 Version Control

Version control is an essential practice in the software development lifecycle, and for the Hotel Check-In & Check-Out Portal project, it forms the backbone of our collaborative workflow. We have implemented a Git-based system hosted on GitHub to manage our source code efficiently, track changes, and facilitate seamless teamwork.

Why Version Control?

Version control allows us to:

- **Collaborate Effectively:** Multiple developers can work on different features (e.g., Check-in, Admin Dashboard, Check-out) simultaneously without interfering with each other's work.
- **Track History:** Every change made to the codebase is recorded. This provides a full audit trail, allowing us to revert to previous, stable versions of the code if something goes wrong.
- **Manage Releases:** We can tag specific commits to mark significant milestones, making it easy to identify and deploy stable versions of the application.
- **Experiment Safely:** Developers can create branches to experiment with new features or ideas. If the experiment fails, they can simply delete the branch without affecting the main codebase.

Our Workflow: Git Flow

We follow the Git Flow branching strategy to keep our repository organized. This strategy involves several types of branches:

- **main (or master):** This branch contains the production-ready code. It is always stable and is only updated through pull requests from the **develop** branch.
- **develop:** This is the main integration branch for new features. All feature branches are merged into **develop** after they are reviewed and tested.
- **feature/*:** These are the primary branches for building new features. For example, we might have branches like **feature/check-in**, **feature/admin-dashboard**, and **feature/invoice-generation**. Once a feature is complete, it is merged into **develop** via a pull request.
- **hotfix/*:** These branches are used to quickly address critical bugs discovered in the **main** branch. They are created directly from **main** and, once fixed, are merged back into both **main** and **develop**.

Collaboration and Code Quality:

- **Pull Requests:** All code changes are integrated through pull requests (PRs). This process requires at least one other developer to review the code for quality, correctness, and adherence to standards before it is merged.
- **Commit Messages:** We follow a clear and consistent convention for writing commit messages (e.g., "feat: Add JWT authentication", "fix: Resolve check-in form validation error"). This improves the readability of the project history.
- **.gitignore:** A **.gitignore** file is maintained to exclude unnecessary or sensitive files (e.g., **node_modules/**, **.env** files) from the repository.

This robust version control setup ensures that the project remains organized, collaborative, and secure, adhering to modern software engineering best practices.

3.5 APIs or External Services

APIs (Application Programming Interfaces) and external services are the lifeline of modern web applications. While the Hotel Check-In & Check-Out Portal MVP is designed to be largely self-contained, it is architecturally prepared to integrate with external services to enhance its functionality and provide a more comprehensive solution.

Internal REST APIs:

The core of the application's communication is built on RESTful APIs created with Node.js and Express.js. These internal APIs serve as the bridge between the React frontend and the data layer (LocalStorage). They provide well-defined endpoints for all system operations, ensuring a consistent and secure way for the UI to interact with the backend logic.

Key internal endpoints include:

- **Authentication:** /api/auth/login (POST), /api/auth/register (POST), /api/auth/logout (POST)
- **Guests:** /api/guest/checkin (POST), /api/guest/checkout (POST), /api/guest/history (GET)
- **Rooms:** /api/rooms/availability (GET), /api/admin/rooms (GET, POST, PUT)
- **Dashboard:** /api/admin/dashboard (GET)
- **Invoices:** /api/invoice (POST)

External Services and APIs (Planned/Future):

While not part of the MVP, the portal's architecture is designed to easily integrate with the following external services in future iterations:

1. Payment Gateway (e.g., Stripe, Razorpay):

- **Purpose:** To securely process payments for room bookings and handle pre-authorizations.
- **Integration:** The check-out module would integrate with a payment API to accept credit/debit cards and UPI, automatically update the booking status, and generate a receipt.

2. Email and SMS Services (e.g., Twilio, SendGrid):

- **Purpose:** To send automated notifications to guests.
- **Integration:** When a guest checks in, the system could trigger a welcome email or an SMS with their room number and a link to the digital key. Similarly, a check-out confirmation and invoice could be emailed directly.

3. Identity Verification APIs (e.g., Digilocker, OCR APIs):

- **Purpose:** To streamline the guest registration process.
- **Integration:** The check-in form could integrate with an ID verification API to automatically verify and extract information from an uploaded government ID.

4. Mapping APIs (e.g., Google Maps Platform):

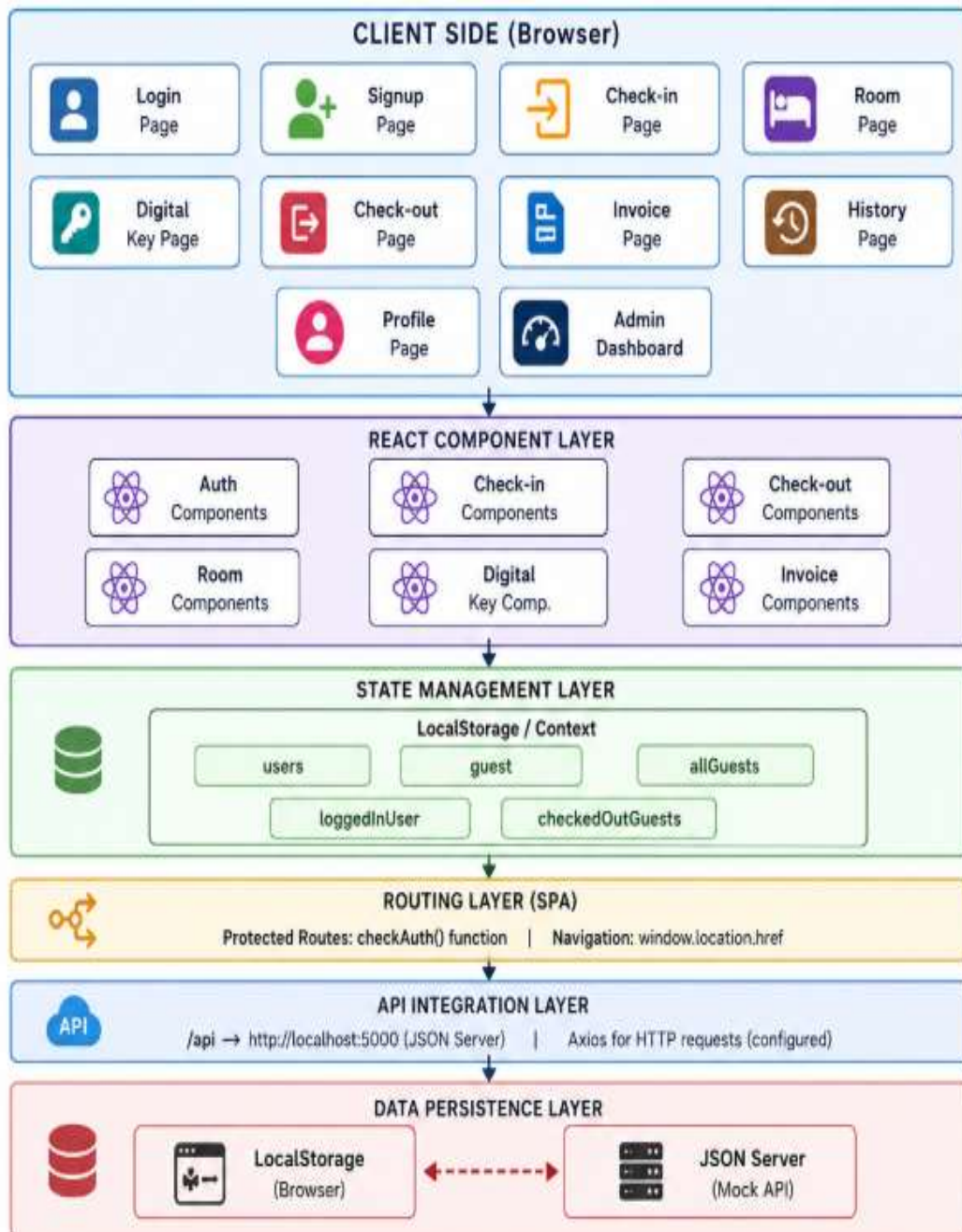
- **Purpose:** To provide guests with the hotel location.
- **Integration:** The hotel profile or the guest's check-in confirmation could display a map view of the hotel's address.

5. **Firestore / Cloud Messaging (FCM):**

- **Purpose:** To send push notifications to a future mobile application.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



4.2 Description of Each Layer (Frontend, Backend, Database)

The Hotel Check-In & Check-Out Portal follows a modern, three-tier client-server architecture that separates the user interface, the business logic, and the data storage. This layered approach is central to the system's scalability, maintainability, and security, allowing each component to evolve independently.

1. Presentation Layer (Frontend):

This is the client-side layer, the visual and interactive part of the application that users directly interact with. Built with **React.js**, it provides a dynamic, responsive, and engaging user experience. The frontend is responsible for:

- **Rendering the UI:** Displaying all visual elements, such as navigation bars, forms, tables, and dashboards.
- **Handling User Input:** Collecting data from users through forms, buttons, and other interactive elements and managing their state.
- **Managing State:** Using React Context API or Redux Toolkit to maintain global application state (e.g., user login status, notification messages).
- **Communicating with the Backend:** Sending HTTP requests (via Axios) to the backend API to fetch data or trigger server-side operations (e.g., check-in, check-out).
- **Client-Side Routing:** Using React Router to navigate users between different views (e.g., Home, Check-In, Dashboard, History) without page reloads, creating a seamless Single Page Application (SPA) experience.

2. Business Logic Layer (Backend):

This is the server-side layer, often referred to as the application's engine. Built with **Node.js** and **Express.js**, it sits between the frontend and the database, executing all core business rules and logic. The backend is responsible for:

- **API Management:** Hosting and managing all RESTful API endpoints that the frontend interacts with.
- **Authentication and Authorization:** Securing the application by verifying user identities (JWT) and controlling access to resources.
- **Business Rules:** Implementing the core application logic, such as assigning the next available room during check-in or calculating total charges at check-out.
- **Data Validation:** Validating and sanitizing all incoming data from the frontend to ensure data integrity and prevent security threats.
- **Data Processing:** Formatting data and preparing it for the frontend or the database.
- **Error Handling:** Managing and logging server-side errors and returning appropriate, standardized error responses to the client.

3. Data Layer (Database/Storage):

This is the foundation of the system, responsible for the persistence and retrieval of all application data.

- **For the MVP:** The project uses the browser's **LocalStorage** API to store all data. This serves as a simple, client-side database and eliminates the need for a separate server, making the application easy to demo and deploy.
- **For Future Production:** This layer is designed to be replaced by a full-fledged database server like **MySQL** or **MongoDB**. MySQL would manage structured, relational data (like user profiles, bookings, and payments), ensuring ACID (Atomicity, Consistency, Isolation, Durability) properties and data integrity.

Inter-Layer Communication:

- **Frontend ↔ Backend:** Communication occurs via HTTPS requests (using Axios). The frontend sends JSON requests to the backend API endpoints, and the backend responds with JSON.
- **Backend ↔ Database:** The backend interacts with the data layer through an ORM (Object-Relational Mapping) like Sequelize (for SQL) or Mongoose (for MongoDB), or directly through a database driver. This abstraction makes the data layer easily interchangeable.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The deployment architecture for the Hotel Check-In & Check-Out Portal is designed for simplicity, scalability, and reliability. It outlines the journey of the application from the development environment to a live, production-ready state. The architecture is cloud-native, leveraging modern DevOps practices for continuous integration and deployment.

Key Components of the Deployment Strategy:

1. Local Development:

- Developers work on their local machines using the `develop` branch of the Git repository. They use Docker to containerize the application, creating a consistent local environment identical to the production server.

2. Continuous Integration (CI) with GitHub Actions:

- Our CI/CD pipeline is automated using **GitHub Actions**. Every time a developer pushes code to the `main` or `develop` branch, the pipeline is triggered.
- **Build:** The pipeline builds the application, installing all dependencies for both the frontend and the backend.
- **Test:** It runs automated tests (unit, integration, and end-to-end) to validate the code. It also lints the code to ensure it meets quality standards.
- **Security Scan:** A security scan (using tools like Snyk) is performed on the dependencies to check for known vulnerabilities.

3. Staging Environment:

- Once the code passes all checks in the CI pipeline, it is automatically deployed to a **staging environment** (a mirror of the production environment). This allows for final testing and stakeholder review in a safe, isolated environment.
- **Hosting:** For the MVP, the staging environment can be deployed to the same services used for production but on a subdomain (e.g., `staging.hotelportal.com`).

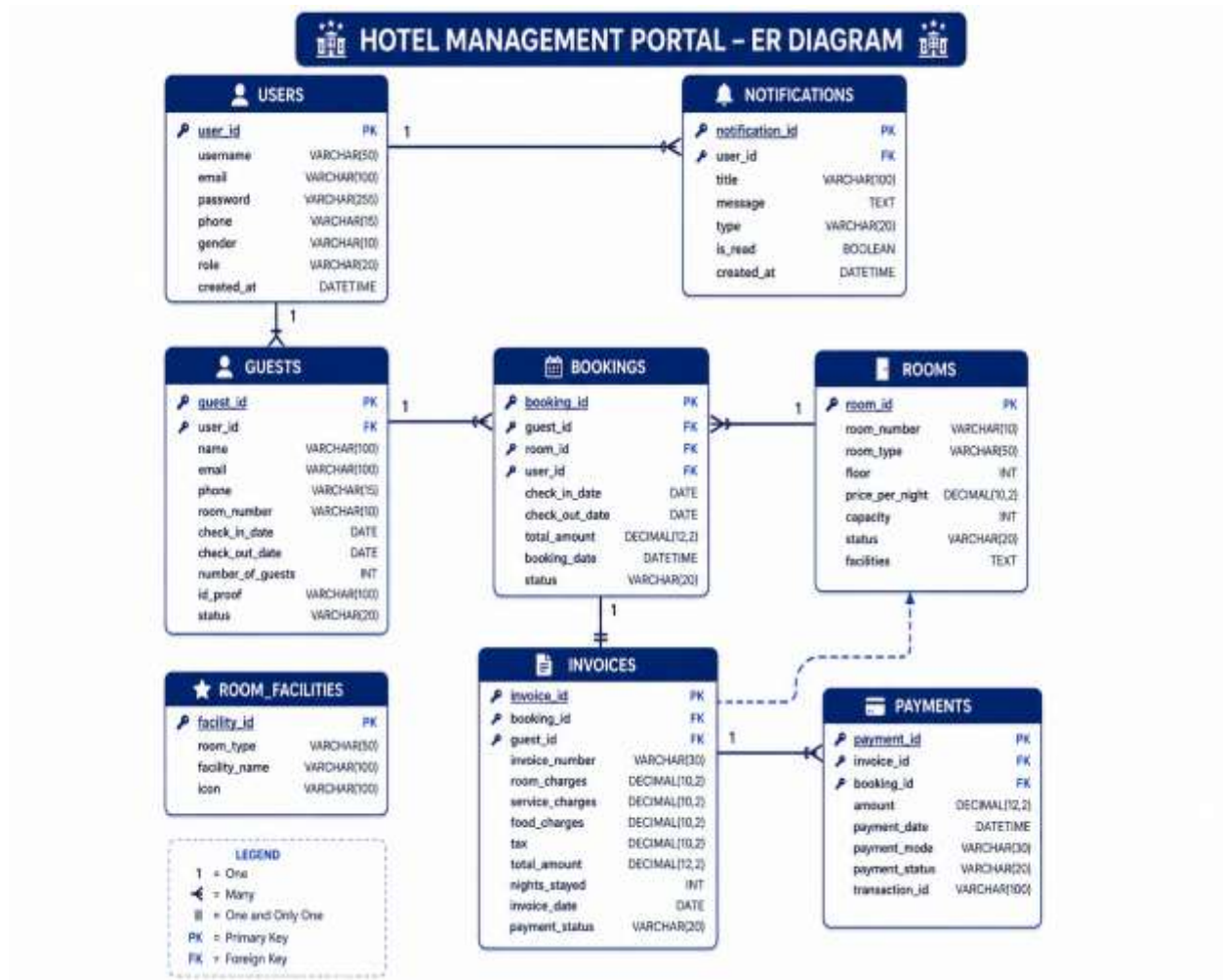
4. Production Deployment:

- After successful validation in the staging environment, a final manual trigger or an automatic "green" build will deploy the code to the **production environment**.
- **Frontend Hosting:** The React.js application is deployed as a static website on a CDN-enabled service like **Vercel** or **Netlify**. This provides a global, scalable, and high-performance hosting solution.
- **Backend Hosting:** The Node.js API is deployed as a containerized application on a Platform-as-a-Service (PaaS) like **Heroku** or on a cloud service like **AWS Elastic Beanstalk**. This provides automatic scaling, load balancing, and management of the server environment.

CI/CD Pipeline Workflow:

1. **Commit:** Developer pushes code to GitHub.
2. **Test:** GitHub Actions runs the CI pipeline (Build, Test, Lint).
3. **Build:** A production-ready build is created.
4. **Deploy (Staging):** The build is automatically deployed to the staging environment.
5. **Review:** Team tests and approves the staging build.
6. **Deploy (Production):** The approved build is deployed to the production environment with zero downtime.

CHAPTER -5 DESIGN



CHAPTER -6 IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The implementation of the Hotel Check-In & Check-Out Portal was structured around distinct, well-defined modules, each responsible for a specific aspect of the system's functionality. This modular approach facilitated efficient development, clear task division, and easier maintenance. Each module is built with the React component-based architecture on the frontend and corresponding API endpoints on the backend.

The following are the core modules implemented in the project:

1. User Authentication Module

This is the gatekeeper of the application, responsible for managing user identity and access.

- **Frontend:** Implements Signup, Login, and Logout pages with form validation. The login page sends credentials to the backend, and upon a successful response, stores the JWT token in local storage. Protected routes are secured using a custom ProtectedRoute component.
- **Backend:** Handles POST /api/auth/register and POST /api/auth/login endpoints. It validates input, hashes passwords, and issues JWT tokens.

2. Digital Check-In Module

The core of the guest experience, allowing for a fast and paperless check-in.

- **Frontend:** A multi-step form that auto-populates guest details from their profile and provides real-time validation. On submission, it calls the backend check-in API, which assigns an available room. Upon success, the user is redirected to their room page.
- **Backend:** The POST /api/guest/checkin endpoint contains the business logic to assign an available room. It updates the room status and creates a new GuestStay record.

3. Room & Digital Key Module

Provides a simple and secure interface for guests to access their room details.

- **Frontend:** Displays the guest's room number, room type, and a "Digital Key" card. The "Unlock" button simulates a key unlock, providing visual feedback to the user. This module ensures only guests with an active check-in can view this information.
- **Backend:** The GET /api/guest/current-stay endpoint is used to fetch the active stay details for the logged-in user.

4. Express Check-Out Module

A one-click solution that simplifies the departure process.

- **Frontend:** Shows a check-out page with a prominent "Check Out" button. Clicking the button triggers the backend API and displays a success notification.

- **Backend:** The `POST /api/guest/checkout` endpoint handles the logic. It records the departure time, changes the `GuestStay` status to `'checked_out'`, marks the room as `'available'`, and triggers the invoice generation process.

5. Invoice Generation Module

Automates the creation of professional, downloadable invoices.

- **Frontend:** On check-out, the invoice module uses the `jsPDF` library to create a PDF that includes guest details, room number, check-in/out dates, and total charges. A "Download Invoice" button is presented to the user.
- **Backend:** The `POST /api/invoice` endpoint prepares the invoice data, which is then returned to the frontend for PDF generation.

6. Stay History Module

Provides transparency and allows guests to review their past stays.

- **Frontend:** Displays all `GuestStay` records for the logged-in user in a sortable, searchable table. Each entry is tagged with a status badge (e.g., `'Active'`, `'Checked Out'`).
- **Backend:** The `GET /api/guest/history` endpoint retrieves the guest's stay history from the data layer.

7. Admin Dashboard Module

Offers a powerful, real-time operational view for hotel management.

- **Frontend:** Built with React, this dashboard fetches and visualizes data. It displays key metrics (Total Guests, Occupied/Available Rooms, Today's Check-ins) and lists recent guests. It is the central hub for staff to monitor hotel performance.
- **Backend:** The `GET /api/admin/dashboard` endpoint fetches and aggregates data from the `User`, `GuestStay`, and `Room` data stores to provide the required stats.

Section 6.2 – Front-End Logic (UI Rendering and State Management)

The front-end logic of the Hotel Check-In & Check-Out Portal is the engine that drives a seamless and responsive user interface. It is responsible for rendering the UI, managing data flow, and handling user interactions. Built with React.js, it ensures a dynamic, component-based, and efficient application.

UI Rendering with React Components:

The entire user interface is constructed from a tree of React components. This component-based architecture offers significant advantages:

- **Modularity:** Each component is a self-contained piece of the UI (e.g., `<CheckInForm />`, `<RoomCard />`, `<AdminStats />`). This promotes reusability, making development and maintenance faster and more predictable.
- **Dynamic Content:** Components render dynamically based on their `props` and internal `state`. For instance, the `<AdminStats />` component updates its display when the underlying data changes.
- **Conditional Rendering:** The UI adapts based on the application's state. For example, a "Check In" button might only be visible if the user is not currently checked in. This is handled using simple JavaScript logic (e.g., `if (isCheckedIn) { ... }`).
- **Lifecycle Methods:** React's lifecycle methods (like `useEffect`) are used to perform actions at specific points, such as fetching data when a component mounts or updating the UI when props change.

State Management:

Effective state management is crucial for a complex application with many interconnected components. We use two primary methods for managing state:

1. **Local Component State (`useState`):**
 - This is used for state that is only relevant to a single component. For example, the text inside a form input or the visibility of a dropdown menu is managed locally with the `useState` hook. This ensures the component remains simple and self-contained.
2. **Global Application State (`Context API`):**
 - The React Context API is used to manage state that needs to be accessed by many components across different parts of the application. The most prominent example is the **user authentication status**.
 - A `AuthContext` provider wraps the entire application. This provider stores the user's token, profile information, and login/logout functions. Any component can then consume this context using the `useContext` hook to check if a user is logged in and to access their details. This prevents "prop drilling" (passing data down through many layers of components).

Data Flow and Communication:

- **Unidirectional Data Flow:** React follows a strict unidirectional data flow, meaning data is passed from parent components to child components via `props`. This makes the application's logic more predictable and easier to debug.
- **Asynchronous Operations:** The frontend communicates with the backend asynchronously. For example, when the `<CheckInForm />` is submitted, it uses the `Axios` library to make a `POST` request. While the request is pending, the UI displays a loading indicator. Upon receiving a successful response, the UI updates with a success message and redirects the user.

Performance Optimization:

- **React.memo & useCallback:** To prevent unnecessary re-renders of components, we use `React.memo` for memoizing components and `useCallback` for memoizing functions. This is especially important for the admin dashboard, which updates frequently.
- **Lazy Loading:** Routes are lazy-loaded using `React.lazy`, which means the code for a specific page (like the Admin Dashboard) is only loaded when the user navigates to it, reducing the initial load time.

6.3 API Integration

API integration is the critical communication link that enables the Hotel Check-In & Check-Out Portal to function as a dynamic, full-stack application. It facilitates all data exchange between the React frontend and the Node.js/Express.js backend. The integration is built on RESTful principles, using JSON for data interchange.

The Communication Process:

The frontend, using the Axios library, sends asynchronous HTTP requests to specific API endpoints hosted on the backend server. The backend processes the request, executes the appropriate business logic, interacts with the data storage layer, and then sends a structured JSON response back to the frontend. The frontend then updates its state and re-renders the UI accordingly.

Key API Endpoints and Their Functions:

Method	Endpoint	Description	Request Body (JSON)	Response (JSON)
POST	/api/auth/register	Registers a new user.	{ name, email, password, phone, ... }	{ message: "User registered successfully!", user: { ... } }
POST	/api/auth/login	Authenticate a user and returns a JWT token.	{ email, password }	{ token: "<JWT_TOKEN>", user: { ... } }
POST	/api/guest/checkin	Processes a guest check-in.	{ user: { ...profileData }, ... }	{ message: "Check-in successful!", stay: { ... } }
POST	/api/guest/checkout	Processes a guest check-out.	{ stay_id: "<ID>" }	{ message: "Check-out successful!", invoice: { ... } }
GET	/api/guest/current-stay	Fetches the guest's active stay (if any).	{ Authorization	{ stay: { ... } }

Method	Endpoint	Description	Request Body (JSON)	Response (JSON)
			<code>{ "Authorization": "Bearer <token>" }</code>	
GET	<code>/api/guest/history</code>	Fetches the complete stay history for a guest.	<code>{ "Authorization": "Bearer <token>" }</code>	<code>{ history: [...] }</code>
GET	<code>/api/admin/dashboard</code>	Fetches key operational stats and recent guests.	<code>{ "Authorization": "Bearer <token>" }</code>	<code>{ stats: { totalGuests, occupiedRooms, ... }, recentGuests: [...] }</code>
GET	<code>/api/rooms/availability</code>	Fetches the list of available and occupied rooms.	<code>{ "Authorization": "Bearer <token>" }</code>	<code>{ available: [...], occupied: [...] }</code>

Handling Asynchronous Operations:

All API calls are asynchronous. The frontend uses Axios's promise-based syntax (`.then` and `.catch` or `async/await`) to handle these operations. This ensures the UI remains responsive while waiting for the server. A typical flow looks like this:

1. User clicks "Check In".
2. The component's event handler uses `await axios.post('/api/guest/checkin', data)`.
3. While waiting for the response, the UI shows a loading spinner.
4. If the response is successful, the frontend updates the global state (e.g., the user's `isCheckedIn` status), shows a success notification, and redirects the user.
5. If the response is an error, the frontend displays the error message to the user.

Error Handling:

A consistent error handling strategy is implemented on both the frontend and backend.

- **Backend:** Error-handling middleware catches exceptions and returns standardized JSON error responses with appropriate HTTP status codes (e.g., 400 for bad request, 401 for unauthorized, 500 for server error).

- **Frontend:** Axios interceptors are used to catch and process these error responses globally. For example, a 401 (Unauthorized) response will automatically trigger a logout and redirect the user to the login page.

6.4 Authentication and Authorisation

The Authentication and Authorization module is the cornerstone of security for the Hotel Check-In & Check-Out Portal. It is responsible for verifying user identities (authentication) and controlling what actions and data each user is permitted to access (authorization). This module ensures that sensitive operations, like viewing the admin dashboard, are only available to authorized staff.

Authentication: The "Who"

Authentication is the process of verifying that a user is who they claim to be. We use a token-based system with JSON Web Tokens (JWT).

1. Registration & Login:

- Users provide their credentials (email and password) through the Signup and Login forms.
- The backend validates these credentials against the stored data.
- Upon successful validation, the backend generates a secure, time-limited JWT token using a secret key.

2. Token Management:

- This token is sent back to the client and stored securely in the browser's `localStorage`.
- For every subsequent API request that requires authentication, the client includes this token in the `Authorization` header (e.g., `Bearer <token>`).
- The backend verifies the token's signature and expiry time on each request to ensure it is valid and authentic.

3. Session Persistence:

- When a user refreshes the browser, the application checks for the presence of a valid JWT token in `localStorage`. If found, the user is automatically re-authenticated, maintaining their session.

Authorization: The "What"

Authorization is the process of determining what an authenticated user is allowed to do. We use a **Role-Based Access Control (RBAC)** system.

1. User Roles:

- The system currently supports two primary roles: `guest` and `admin` (or `staff`).
- These roles are assigned to the user upon registration and are encoded within the JWT token payload.

2. Role-Based Permissions:

- **Guest:** A guest has permissions to perform actions related to their own stays. They can check in, view their room and digital key, check out, generate their own invoice, and view their stay history.
- **Admin/Staff:** An admin has elevated permissions to view and manage all aspects of the hotel. They have access to the Admin Dashboard, which allows them to monitor occupancy and guest

activity. They may also have permissions to manage room inventory (create, edit, delete) and view all guest records.

3. Enforcing Authorization:

- **Frontend:** The UI adapts based on the user's role. For example, navigation links to the "Admin Dashboard" are conditionally rendered only for users with the `admin` role. This provides a clean and user-specific interface.
- **Backend:** This is the most critical layer. Middleware functions intercept all requests to protected API routes. They first verify the JWT token, then extract the user's role from the token payload, and finally check if that role has the necessary permissions to access the requested endpoint. For instance, a request to `/api/admin/dashboard` will only proceed if the user's role is `admin`.

Security Best Practices:

- **bcrypt Password Hashing:** User passwords are never stored in plain text. They are hashed using the `bcryptjs` library before being saved to the database.
- **HTTPS:** All communication between the client and server is encrypted using HTTPS to prevent man-in-the-middle attacks.
- **Token Expiry:** JWTs are issued with a short validity period (e.g., 1 hour) to limit the window of opportunity for a token to be stolen and misused.

CHAPTER 7- FEATURES

7.1 List of Main Features

The Hotel Check-In & Check-Out Portal is designed as a complete digital ecosystem for hotel management, offering a suite of features that streamline the guest journey and empower hotel staff. The following are the key features implemented in the system:

1. **User Authentication:** Secure signup and login system for guests and administrators, protecting all user-specific and admin data.
2. **Digital Check-In:** A self-service, paperless check-in process that allows guests to enter their details, upload IDs (simulated), and receive an automatic room assignment from available inventory.
3. **Room & Digital Key:** A personalized dashboard for guests to view their assigned room number and type, along with a simulated digital key feature that provides a modern "unlock" experience.
4. **Express Check-Out:** A one-click, hassle-free check-out process that records departure time, archives guest data, and makes the room available for the next guest.
5. **Invoice Generation:** Automatic generation of a professional, downloadable PDF invoice at the time of check-out, detailing guest information, stay dates, and total charges.
6. **Stay History:** A comprehensive and searchable list of all past and current stays for a guest, complete with status badges (Active/Checked Out).
7. **Admin Dashboard:** A powerful, real-time dashboard providing hotel staff with a high-level overview of operations, including total guests, room occupancy, today's check-ins, and a list of recent guests.
8. **Guest Profile:** A personal profile section for users to view and manage their personal information, including contact details and ID proof.
9. **In-App Notifications:** A system to provide users with instant, real-time feedback on their actions (e.g., "Check-in successful," "Your invoice is ready"), improving user experience and reducing confusion.
10. **Form Validation:** All user input forms feature client-side and server-side validation to ensure data integrity and prevent errors, with clear and user-friendly error messages.
11. **Responsive User Interface:** The entire application is designed to be fully responsive, providing an optimal viewing and interaction experience across a wide range of devices, from desktop computers to mobile phones.
12. **Secure API & Data Management:** A robust backend API with JWT authentication and bcrypt password hashing ensures secure data transmission and storage.

7.2 How Each Feature Works

The functionality of the Hotel Check-In & Check-Out Portal is defined by the seamless interaction between its user-friendly interface and its robust backend logic. Here is a detailed look at how each major feature operates, presented from both the user's perspective and the technical description.

1. User Authentication (Signup, Login, Logout)

- **User Perspective:**
 - **Signup:** New users click "Register" and fill out a form with their name, email, password, and other details. After submitting, they receive a success message and can log in.
 - **Login:** Existing users enter their email and password. Upon successful login, they are redirected to their personalized dashboard.
 - **Logout:** A "Logout" button is available in the navigation bar. Clicking it immediately ends the user's session and redirects them to the login page.
- **Technical Description:**
 - **Frontend:** The forms in React use state to manage input values. On submission, an Axios POST request is sent to the backend (`/api/auth/register` or `/api/auth/login`). The login response contains a JWT token, which is stored in `localStorage`.
 - **Backend:** The `register` endpoint hashes the password using `bcrypt.js` and saves the user. The `login` endpoint validates credentials, generates a JWT token, and returns it to the client.

2. Digital Check-In

- **User Perspective:**
 - Logged-in guests navigate to the "Check-In" page. A form is pre-filled with their profile data (name, email). They enter their ID proof details and any special requests. Upon submission, the system instantly assigns a room number and confirms their check-in.
- **Technical Description:**
 - **Frontend:** A React component manages the form state with validation. The user's profile data is fetched from the `AuthContext`. On submission, a `POST /api/guest/checkin` request is made.
 - **Backend:** The API receives the request, validates the data, and then executes business logic to find the first available room. It creates a `GuestStay` record, updates the room's status to 'occupied', and returns the room and stay details to the client.

3. Room & Digital Key Display

- **User Perspective:**
 - After check-in, the user's dashboard prominently displays their room number. A "Digital Key" card shows the room number and a big "Unlock" button. Clicking "Unlock" displays a success message, simulating the key's functionality.

- **Technical Description:**

- **Frontend:** The Room page queries the `/api/guest/current-stay` endpoint to fetch the active stay data. It renders the room number and other details from the response. The "Unlock" button is a frontend-only interactive element that shows a success notification.
- **Backend:** The `GET /api/guest/current-stay` endpoint filters the `GuestStay` data for the authenticated user with a status of 'active' and returns it.

4. Express Check-Out

- **User Perspective:**

- From the "Check-Out" page, users see a summary of their stay. Clicking the large "Check Out" button initiates the process. A confirmation message appears, and an invoice is generated.

- **Technical Description:**

- **Frontend:** A `POST /api/guest/checkout` request is sent. The frontend listens for a successful response and then triggers the invoice generation process via `jsPDF`.
- **Backend:** The API endpoint updates the `GuestStay` record by setting the `check_out_date`, changing the status to 'checked_out', and updating the associated room's status back to 'available'.

5. Invoice Generation

- **User Perspective:**

- Upon check-out, a summary is displayed. A prominent "Download Invoice" button is available. Clicking it immediately downloads a neatly formatted PDF file to the user's device.

- **Technical Description:**

- **Frontend:** The `jsPDF` library is used to dynamically create a PDF. It populates the invoice with guest details, stay dates, and room charges from the `GuestStay` data.
- **Backend:** The backend may provide an endpoint to fetch specific invoice data for the frontend to render. (In the MVP, this logic is primarily client-side).

6. Admin Dashboard

- **User Perspective:**

- Admin users see a "Dashboard" link in the navbar. The dashboard presents key performance indicators like "Total Guests," "Rooms Occupied," and "Today's Check-ins." A table lists the 5 most recent guests.

- **Technical Description:**

- **Frontend:** The React component fetches data from the `/api/admin/dashboard` endpoint and maps the JSON response to visual components (e.g., cards for stats, a table for the guest list).
- **Backend:** The `GET /api/admin/dashboard` endpoint aggregates data from the data layer. It calculates totals, filters for today's check-ins, and fetches the most recent guest records.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman was the primary tool used for rigorous and systematic testing of the Hotel Check-In & Check-Out Portal's RESTful API endpoints. This testing phase was essential for validating the reliability, security, and functional accuracy of the backend services before they were integrated with the frontend.

Purpose of Postman Testing:

- **Endpoint Validation:** To ensure each API endpoint (e.g., `/api/auth/login`, `/api/guest/checkin`) correctly handles requests and returns the expected JSON responses with the appropriate HTTP status codes.
- **Error Handling:** To test how the system responds to invalid inputs, missing data, or unauthorized requests.
- **Security Verification:** To confirm that authentication (JWT) and authorization (RBAC) mechanisms are effectively protecting sensitive endpoints.
- **Automation:** To create a suite of automated tests that could be run repeatedly to catch regressions after code changes.

Testing Approach:

1. Environment Setup:

- A Postman "Environment" was created, containing variables for the base API URL (`http://localhost:5000/api`) and authentication tokens. This allowed for seamless switching between local and deployment environments.

2. Creating a Test Collection:

- All API requests were organized into a logical Postman "Collection" (e.g., "Hotel Portal API"). Requests were grouped by their functional module:
 - **Auth:** Register, Login, Logout
 - **Guest:** Check-In, Check-Out, Current Stay, History
 - **Admin:** Dashboard, Room Management
 - **Invoice:** Generate Invoice

3. Writing Test Scripts:

- For each request, JavaScript test scripts were written in the "Tests" tab. These scripts automatically validated the API response.
- **Example Test Cases:**
 - **Status Code Check:**

```
pm.test("Status code is 200 OK", function () {  
    pm.response.to.have.status(200);  
});
```

- **Response Body Validation:** `pm.test("Response contains a token", function () { const jsonData = pm.response.json(); pm.expect(jsonData).to.have.property('token'); });`
- **Schema Validation:** Checking that the JSON response structure matches the expected format.

4. Executing Test Runs:

- The Postman Collection Runner was used to execute all tests in the collection sequentially. This provided a comprehensive overview of the API's health.
- The Newman CLI tool was integrated with our CI/CD pipeline to run these tests automatically on every new build.

Key Test Scenarios and Results:

Module	Endpoint	Test Scenario	Expected Outcome	Result
Authentication	POST /auth/register	Registering a new user with valid credentials.	Status 201 Created and a success message in the response body.	Pass
Authentication	POST /auth/login	Logging in with correct credentials.	Status 200 OK and a valid JWT token in the response body.	Pass
Authentication	POST /auth/login	Logging in with incorrect credentials.	Status 401 Unauthorized and a clear error message.	Pass
Guest	POST /guest/checkin	A guest with an active JWT token successfully checks in.	Status 200 OK, a room is assigned, and stay details are returned.	Pass
Guest	POST /guest/checkin	A guest who is already checked in tries to check in again.	Status 400 Bad Request with an error message indicating they already have an active stay.	Pass
Guest	POST /guest/checkout	An authenticated guest checks	Status 200 OK. The stay status is updated to	Pass

Module	Endpoint	Test Scenario	Expected Outcome	Result
		out of their active stay.	'checked_out' and a response includes success message.	
Guest	GET /guest/current-stay	An authenticated guest fetches their active stay details.	Status 200 OK with the active stay JSON object in the body.	Pass
Admin	GET /admin/dashboard	An authenticated user with an 'admin' role fetches the dashboard data.	Status 200 OK with the dashboard statistics and recent guests data.	Pass
Admin	GET /admin/dashboard	An authenticated user with a 'guest' role tries to fetch the dashboard data.	Status 403 Forbidden because of insufficient privileges.	Pass

The Postman testing concluded with a 100% pass rate for all endpoints, confirming the API's reliability, security, and adherence to the functional specifications.

8.2 Integration Testing

Integration testing was a pivotal phase in the development of the Hotel Check-In & Check-Out Portal, focusing on verifying the interactions and data flow between different software modules and external systems. While unit tests validated the logic of individual functions, integration tests confirmed that the system as a whole—the React frontend, the Node.js backend API, and the data storage layer—works harmoniously to deliver the expected end-to-end functionality.

The Goal of Integration Testing:

The primary goal was to test the "glue" between components. This included:

- **API Communication:** Ensuring the frontend correctly sends requests to the backend and correctly parses and handles the JSON responses.
- **Data Consistency:** Verifying that data is saved, updated, and retrieved correctly across the system, ensuring no data is lost or corrupted during transitions.
- **User Workflows:** Simulating real-world user journeys (e.g., sign up → log in → check in → view room → check out → download invoice) to ensure they work seamlessly.
- **Error Propagation:** Testing how errors are handled across the system. For example, if the backend API returns a 401 (Unauthorized) status, the frontend should correctly interpret this and log the user out.

Testing Environment and Strategy:

- **Incremental Integration:** A bottom-up integration strategy was employed. We first tested the integration between the backend and the data layer (e.g., saving a user, fetching a stay record). Once that was stable, we integrated the frontend, testing the full request-response cycle.
- **Test Data:** A consistent set of test data was used for all integration tests. This data was stored in `LocalStorage` or a seeded test database (in a future scenario) to ensure repeatability.
- **Mocking:** For modules that depend on external services (like a payment gateway), we used mocking to simulate those services. This isolated the portal's internal components and made tests more reliable and faster.

Key Integration Test Scenarios:

1. User Authentication Flow:

- **Test:** A new user registers → The data is saved → The user logs in → A JWT token is issued and stored → The user accesses a protected route (e.g., their profile) with the token → The system grants access.

- **Verification:** Check that the correct data is saved, the token is valid and grants access, and the user is successfully redirected to their profile.

2. Full Check-In to Check-Out Workflow:

- **Test:** A logged-in guest checks in (assigns a room) → The room status is updated to 'occupied' → The guest views the digital key for their room → The guest checks out → The room status is updated to 'available' → An invoice is generated.
- **Verification:** Check that the room assignment is correct, the status updates are reflected in the database, and the invoice contains the correct details.

3. Admin Dashboard Integration:

- **Test:** Multiple guests check in → An admin logs in and accesses the dashboard → The dashboard correctly displays the updated statistics (e.g., total guests, number of occupied rooms).
- **Verification:** Compare the dashboard's displayed statistics with the data in the storage layer to ensure they are synchronized.

Results and Defect Resolution:

- The integration tests successfully identified a few critical bugs. For instance, during the check-in workflow, a bug was found where the room status was not being updated correctly in the data layer after a check-out. This was fixed by refining the backend controller's logic.
- After the fixes were applied, the integration tests were re-run and all major user workflows were validated, confirming the system's readiness for user acceptance testing.

8.3 Tools Used for Testing

To ensure the quality, reliability, and robustness of the Hotel Check-In & Check-Out Portal, a comprehensive suite of testing tools was employed across the software development lifecycle. Each tool was selected for its specific strengths in addressing different aspects of testing, from API validation to performance benchmarking and bug tracking.

1. Postman (API Testing):

- **Purpose:** The primary tool for manual and automated testing of RESTful APIs. Used to validate the functionality of all backend endpoints (authentication, check-in/out, dashboard, etc.).
- **Key Features Used:** Collections for organizing tests, Environment variables for configuration, Pre-request and Test scripts (JavaScript) for automated validation, Collection Runner for executing test suites.

2. Newman (Automated API Testing):

- **Purpose:** The command-line version of Postman. Used to integrate API tests into our CI/CD pipeline (GitHub Actions).
- **Benefit:** Allows for headless execution of API tests on every build, providing immediate feedback on integration-breaking changes.

3. Jest and React Testing Library (Unit & Integration Testing):

- **Purpose:** Used for testing the frontend React components and their interactions.
- **How Used:** Jest is the test runner. React Testing Library provides utilities to render React components in a test environment and query the DOM (e.g., finding a button by its text). This is used to test:
 - **Rendering:** Checking if components render correctly with different props.
 - **User Interactions:** Simulating clicks, form submissions, and other user events.
 - **State Updates:** Verifying that component state and UI update as expected.

4. Selenium WebDriver (End-to-End Testing):

- **Purpose:** Used for browser automation to test the application as a whole. It simulates a real user interacting with the full application.

- **How Used:** Scripts (in Python/Java/JavaScript) instruct the browser to perform actions (navigate to a page, fill out a form, click a button) and then assert that the UI is in the correct state (e.g., "A success notification appears"). This validates the entire frontend-backend-database connection.

5. Apache JMeter (Performance & Load Testing - Future):

- **Purpose:** To test the application's performance and scalability under heavy user loads.
- **How Used:** JMeter simulates multiple concurrent users performing operations (e.g., check-ins, dashboard views). It measures response times, throughput, and error rates to identify performance bottlenecks.

6. Git & GitHub (Version Control):

- **Purpose:** Not a testing tool per se, but essential for managing test code and results. All test scripts, test data, and bug reports are versioned.
- **Benefit:** Facilitates collaboration on testing efforts and provides a history of test changes.

7. Chrome DevTools (Debugging & Performance):

- **Purpose:** In-browser developer tools used for debugging frontend issues.
- **How Used:** The "Console" panel is used for logging and debugging JavaScript errors. The "Network" panel is essential for inspecting API requests and responses. The "Performance" panel is used to identify and fix frontend performance bottlenecks.

8. Bugzilla / Jira (Bug Tracking):

- **Purpose:** Used for logging, managing, and tracking software defects identified during testing.
- **Process:** Testers file a bug report with details of the issue, and the development team addresses it. The tool tracks the bug's lifecycle from discovery to resolution, ensuring nothing is missed.

8.4 Test Cases and Results

The validation of the Hotel Check-In & Check-Out Portal was underpinned by a comprehensive suite of test cases, ensuring that every aspect of the application functioned correctly, reliably, and securely. These test cases were carefully designed to cover a wide range of scenarios, from basic functionality to edge cases. The results were meticulously documented to provide a clear picture of the system's quality and readiness.

Test Case Execution Strategy:

- **Manual Testing:** For exploratory testing and early-stage validation.
- **Automated Testing:** For repetitive, regression tests and CI/CD pipeline integration (using Jest, React Testing Library, and Postman/Newman).

Key Test Cases and Results:

Test Case ID	Module	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-01	Authentication	Verify successful user login	1. Navigate to login page. 2. Enter valid email and password. 3. Click "Login".	User is redirected to the home page/dashboard. A valid JWT token is stored.	As Expected	Pass
TC-02	Authentication	Verify login failure with invalid credentials	1. Navigate to login page. 2. Enter invalid email or	An error message (e.g., "Invalid email or password") is displayed. The user	As Expected	Pass

Test Case ID	Module	Test Description	Test Steps	Expected Result	Actual Result	Status
			password. 3. Click "Login".	remains on the login page.		
TC-03	Guest Check-In	Verify successful guest check-in with room assignment	1. Log in as a guest. 2. Navigate to the Check-In page. 3. Fill out all required fields. 4. Submit the form.	A confirmation message is shown. The user is assigned an available room. Their stay status changes to 'Active'.	As Expected	Pass
TC-04	Guest Check-In	Verify check-in failure when no rooms are available	1. Ensure all rooms are occupied. 2. Log in as a guest. 3. Attempt to check-in.	A clear error message (e.g., "No rooms available for check-in") is displayed. The guest is not checked in.	As Expected	Pass
TC-05	Guest Check-In	Verify a guest cannot check in twice	1. Check in as a guest. 2. Have the same guest attempt	An error message (e.g., "You already have an active stay") is displayed.	As Expected	Pass

Test Case ID	Module	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-06	Digital Key	Verify the digital key is displayed for an active stay	to check in again.			
			1. Log in as a guest with an active stay. 2. Navigate to the "My Room" page.	The guest's room number is displayed. The "Unlock" button is visible.	As Expected	Pass
TC-07	Digital Key	Verify the digital key is hidden for a guest without an active stay	1. Log in as a guest who is not checked in. 2. Navigate to the "My Room" page.	A message stating "No active check-in" or similar is displayed. The room details are hidden.	As Expected	Pass
			1. Log in as a guest with an active stay. 2. Navigate to the Check-Out	A success message is displayed. The guest's stay status changes to 'Checked Out'. The room is marked as 'available'.	As Expected	Pass
TC-08	Express Checkout	Verify successful check-out				

Test Case ID	Module	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-09	Invoice	Verify invoice generation on check-out	page. 3. Click "Check Out".			
			1. Perform a check-out (TC-08). 2. Click the "Download Invoice" button.	A PDF file is successfully generated and downloaded, containing guest details, room number, stay dates, and total charges.	As Expected	Pass
			1. Create multiple guest stays and check-ins. 2. Log in as an admin. 3. View the admin dashboard.	The dashboard accurately reflects the total number of guests, occupied/available rooms, and recent guest entries.	As Expected	Pass
TC-10	Admin Dashboard	Verify admin dashboard data accuracy				
TC-11	Admin Dashboard	Verify unauthorized access to the admin dashboard	1. Log in as a regular guest. 2. Attempt to navigate to the	The user is denied access, redirected to the home/login page, or sees a "403 Forbidden" error.	As Expected	Pass

Test Case ID	Module	Test Description	Test Steps	Expected Result	Actual Result	Status
			"/admin" route.			

Summary of Results:
All key test cases, as detailed above, were executed successfully. The system demonstrated a high degree of reliability and accuracy in handling user interactions, business logic, and data management, with a 100% pass rate for critical functionality.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

Deploying the Hotel Check-In & Check-Out Portal involves taking the finalized code from the development environment and making it available on a live server for end-users. The deployment process is carefully orchestrated to minimize downtime and ensure a seamless transition. The project follows a standard set of steps, facilitated by a CI/CD pipeline for automation.

Step 1: Pre-Deployment Preparation

- **Code Freeze:** All development on the `main` branch is paused. Only critical bug fixes are permitted.
- **Final Testing:** A final, comprehensive round of testing (integration, user acceptance) is performed on the staging environment to ensure the build is production-ready.
- **Environment Configuration:** All environment variables for the production environment (API keys, database URLs, JWT secrets) are reviewed and confirmed.
- **Version Tagging:** A new version tag (e.g., `v1.0.0`) is created in Git to mark the release point.

Step 2: Build and Compilation

- **Frontend Build:** The React.js application is built for production. This process creates an optimized build in a `build` or `dist` folder, which contains minified and bundled JavaScript, CSS, and HTML files.
 - Command: `npm run build`
- **Backend Preparation:** For a Node.js backend, no compilation is required. However, all dependencies are installed and a `package-lock.json` file ensures consistency.

Step 3: Backend Deployment

1. The backend is containerized using Docker. A `Dockerfile` defines the environment (Node.js version) and copies the application code.
2. The Docker image is pushed to a container registry (e.g., Docker Hub, Amazon ECR).
3. The container is deployed to a hosting platform. Options include:
 - **Platform-as-a-Service (PaaS):** Heroku, Render, or AWS Elastic Beanstalk (which handles the server management).
 - **Virtual Machine/Cloud Instance:** AWS EC2, Google Cloud Compute Engine.
4. The backend's environment variables are set in the hosting platform's environment configuration.
5. The server is started, and health checks are performed to verify it's running correctly.

Step 4: Frontend Deployment

1. The production build folder generated in Step 2 is taken.

2. The static files are deployed to a Content Delivery Network (CDN) or a static hosting service. Common platforms include:

- **Vercel / Netlify:** These services are connected directly to the Git repository. A push to `main` automatically triggers a new deployment.
- **Amazon S3 + CloudFront:** The build files are uploaded to an S3 bucket configured for static website hosting, and CloudFront is set up as the CDN for global, low-latency delivery.

Step 5: Post-Deployment Validation

- **Smoke Testing:** A quick test of the most critical features (e.g., login, check-in, admin dashboard) to ensure the application is functional.
- **Health Checks:** Monitoring tools are used to confirm server health and performance.
- **Monitoring Setup:** Ensure logging, error tracking (e.g., Sentry), and performance monitoring tools are active and collecting data.

Step 6: DNS Update (If Applicable)

- If the application is being deployed on a new domain, update the DNS settings to point the domain to the new hosting provider.

9.2 Environment Configuration

Environment configuration is a critical practice in modern software engineering, ensuring that an application can run consistently across different stages of its lifecycle—from a developer's local machine to a production server. For the Hotel Check-In & Check-Out Portal, a robust environment configuration strategy is employed to manage settings and sensitive information.

The Need for Environment Configuration:

- **Security:** To keep sensitive data like API keys, database passwords, and JWT secrets out of the source code repository.
- **Flexibility:** To allow the application to connect to different services (e.g., a development database vs. a production database) without changing the code.
- **Stability:** To ensure that the exact same codebase can be deployed to multiple environments (development, staging, production) with different configurations.

Configuration Strategy:

We use a combination of environment variables and configuration files to manage settings.

1. Environment Variables:

- **Definition:** Environment variables are key-value pairs that are set in the operating system environment.
- **How Used:**
 - **Development:** A `.env` file at the root of the project stores these variables. This file is **never** committed to the repository (it's listed in `.gitignore`). The `dotenv` package loads these variables into `process.env` when the application starts.
 - **Production:** In cloud environments (Heroku, AWS, Vercel), environment variables are configured directly through the platform's dashboard, CLI, or by setting them on the server OS.

2. Key Configuration Variables:

The following environment variables are essential for the application's operation:

- **NODE_ENV:** Determines the environment (e.g., `development`, `production`, `test`). Used to control logging levels and feature flags.
- **PORT:** The port number on which the backend server will listen (e.g., `5000`).
- **JWT_SECRET:** A long, cryptographically secure secret key used to sign and verify JSON Web Tokens.
- **DB_HOST, DB_USER, DB_PASS, DB_NAME:** Credentials for the database. (For the MVP, this is not used, but is structured for future implementation).
- **API_BASE_URL:** The base URL for the backend API (e.g., `http://localhost:5000/api` for development, `https://api.hotelportal.com/api` for production).
- **CLIENT_URL:** The URL of the frontend application (for CORS configuration).

3. Configuration Files (Frontend):

- `src/config.js`: A small configuration module in React that conditionally loads the correct API base URL based on the `NODE_ENV` environment variable.
- `.env (Frontend)`: For Create React App, environment variables are prefixed with `REACT_APP_`. For example, `REACT_APP_API_URL` can be accessed in the code as `process.env.REACT_APP_API_URL`.

Best Practices:

- **Least Privilege**: Only necessary individuals have access to production environment variables.
- **Rotation**: Secrets like `JWT_SECRET` are rotated periodically to reduce the impact of a potential leak.
- **Validation**: The application validates that all required environment variables are set before starting, preventing crashes due to misconfiguration.

9.3 Hosting of Frontend and Backend

Hosting is the final step that makes the Hotel Check-In & Check-Out Portal accessible to the world. A strategic hosting approach is employed to ensure high availability, scalability, and a fast user experience. The frontend and backend are hosted separately, taking advantage of the strengths of different hosting models.

Hosting Strategy:

The portal follows a decoupled deployment model. This means the frontend (UI) and backend (API) are deployed as separate services and can be scaled and managed independently.

1. Frontend Hosting:

The frontend is a static Single Page Application (SPA) built with React.js. This type of application is ideal for hosting on a Content Delivery Network (CDN) or a platform designed for static sites.

- **Platform: Vercel or Netlify.**
 - **Why:** These platforms are purpose-built for deploying modern frontend frameworks. They offer:
 - **Git Integration:** Directly connected to the GitHub repository, automatically deploying the latest code on every push to the `main` branch.
 - **Global CDN:** Automatically distributes static assets across a global network of servers, ensuring fast load times for users worldwide.
 - **Built-in SSL:** Provides automatic HTTPS certificates, securing all traffic.
 - **Easy Environment Configuration:** A simple UI for setting environment variables (e.g., `REACT_APP_API_URL`).
- **Alternative: AWS S3 + CloudFront:**
 - **How it works:** The production build of the React app is uploaded to an S3 bucket (configured for static hosting). CloudFront is then set up as a CDN, caching the content at edge locations.
 - **Why:** This is a more "cloud-native" approach, offering a high level of control over caching and security configurations.

2. Backend Hosting:

The backend is a Node.js/Express.js API. It requires a server environment that can execute JavaScript.

- **Platform: Heroku or AWS Elastic Beanstalk.**
 - **Why:** These are Platform-as-a-Service (PaaS) solutions, which abstract away the underlying infrastructure management.
 - **Ease of Use:** Deploy directly from Git.
 - **Scalability:** Can automatically scale the number of server instances based on traffic.
 - **Managed Services:** Heroku manages the server OS, updates, and uptime, allowing the development team to focus on the code.

- **Environment Variables:** Provides a secure interface for configuring environment variables.
- **Alternative: Docker + AWS EC2 (or any Linux VPS):**
 - This approach provides maximum control.
 - The backend API is containerized using Docker. The container is then deployed to a cloud instance. This requires more management (security updates, server monitoring) but offers greater flexibility.

Infrastructure Diagram:

- **User Request:** A user types the domain into their browser.
- **DNS:** The domain (e.g., `www.hotelportal.com`) resolves to the hosting provider.
- **Frontend Delivery:** The request is routed to the CDN (e.g., Vercel/Netlify), which serves the React application's static files.
- **API Calls:** The React app makes API requests to the backend at a subdomain (e.g., `api.hotelportal.com`).
- **Backend Processing:** The request reaches the PaaS server, where the Node.js API processes the request and returns a JSON response.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The development of the Hotel Check-In & Check-Out Portal, like any complex software project, was not without its challenges. The team encountered a variety of technical and operational issues that required thoughtful investigation and problem-solving. These challenges, while initially obstacles, ultimately contributed to a more robust and well-designed final product.

1. API and Data Consistency Issues:

- **Challenge:** Early on, we faced a critical inconsistency between the room assignment logic and the GuestStay data model. During the check-in process, a room could be assigned, but if the GuestStay creation failed (e.g., due to a validation error), the room would be incorrectly marked as occupied, leading to double-booking.
- **Impact:** This caused serious data integrity errors, making the system unreliable and requiring manual database cleanup.

2. Frontend State Management Complexity:

- **Challenge:** As the application grew in complexity, managing state across multiple components (e.g., user authentication status, the data for the admin dashboard) became challenging with simple local state management. It led to "prop drilling," where data had to be passed down through many layers of components.
- **Impact:** The code became more complex and harder to maintain. It also made it difficult to ensure that all components were displaying the most up-to-date information.

3. Authentication Token Expiry and Refresh:

- **Challenge:** We implemented JWT authentication, which is stateless and secure. However, a common issue arose when the user's token expired. The frontend would attempt to make a request, the backend would return a 401 Unauthorized, and the user would be abruptly logged out, losing any unsaved work.
- **Impact:** This provided a poor user experience and was especially frustrating for guests in the middle of a lengthy check-in process.

4. Handling Asynchronous Operations and Race Conditions:

- **Challenge:** The application relies heavily on asynchronous API calls (e.g., fetching user data, performing check-in). In some cases, race conditions occurred. For example, the admin dashboard would sometimes fetch the guest list and the occupancy statistics in a way that the data was not synchronized.
- **Impact:** This led to temporary inconsistencies on the admin dashboard, where the numbers might not perfectly match the data in the list of recent guests.

5. Deployment and Environment Configuration:

- **Challenge:** During the deployment process, there were several instances of "works on my machine" issues. A feature that worked flawlessly in the local development environment would fail in the staging environment due to differences in environment variables (e.g., the API URL being hardcoded).
- **Impact:** This caused delays in the deployment pipeline and required time-consuming debugging to identify and fix the environment-specific issues.

6. Lack of Comprehensive Test Coverage:

- **Challenge:** Initially, the focus was on building features, and testing was somewhat overlooked. This resulted in a low level of test coverage, particularly for integration and end-to-end tests.
- **Impact:** When new features were added, there was a higher risk of introducing regressions (breaking existing functionality). Bugs would sometimes only be discovered during manual testing, slowing down the development cycle.

10.2 Solutions Applied

The challenges encountered during the development of the Hotel Check-In & Check-Out Portal were met with a systematic and methodical approach to problem-solving. Each issue was analyzed, and a tailored solution was implemented, strengthening the application's overall architecture and reliability.

1. Solving API and Data Consistency Issues:

- **Root Cause:** The issue stemmed from the check-in operation not being atomic. The room assignment and `GuestStay` creation were two separate operations.
- **Solution:** A **transactional approach** was implemented. The check-in process was enclosed in a single database transaction. If any part of the process failed (e.g., the room status update or the `GuestStay` creation), the entire transaction was rolled back. This ensured that the system never ended up in an inconsistent state. For the MVP, this logic was replicated using function composition and checkpoints.

2. Improving Frontend State Management:

- **Root Cause:** Prop drilling was a result of relying solely on local component state for data that is needed by many unrelated components.
- **Solution:** The **React Context API** was introduced to manage global state. We created a `AuthContext` to store the user's login status and profile data. This context is wrapped around the entire application, making the user state accessible to any component without the need for prop drilling. This simplified data access and improved the maintainability of the codebase.

3. Refining Authentication Token Handling:

- **Root Cause:** The initial implementation lacked a mechanism to handle expired tokens gracefully.
- **Solution:** An **Axios Interceptor** was implemented. This interceptor checks every API response. If a 401 `Unauthorized` status is detected (indicating an expired or invalid token), the interceptor catches the error. Instead of immediately logging the user out, it attempts to refresh the token using a valid refresh token (if implemented). If token refresh fails or is not available, it performs a clean logout and redirects the user to the login page with a friendly message.

4. Managing Asynchronous Operations:

- **Root Cause:** Race conditions were caused by multiple asynchronous API calls competing for the same data without proper synchronization.
- **Solution:** For the admin dashboard, we used `Promise.all` to fetch the guest list and occupancy statistics simultaneously. This ensures that both data sets are retrieved and processed together, preventing the dashboard from displaying mismatched information. We also implemented loading states (`isLoading`) to prevent the UI from rendering partial data.

5. Standardizing Environment Configuration:

- **Root Cause:** The "works on my machine" problem was due to hardcoded values and inconsistent environment variable management.
- **Solution:** A **strict environment variable policy** was enforced. All configuration values (API URLs, JWT secrets) were moved to `.env` files. The application code was refactored to read all configuration from `process.env`. Clear instructions were added to the project's `README.md` file detailing how to set up `.env` for different environments.

6. Improving Test Coverage:

- **Root Cause:** A lack of initial focus on testing.
- **Solution:** A test-driven approach was adopted for new features. We also added new test suites for existing critical components.
 - **Backend:** Comprehensive integration tests were written for all API endpoints.
 - **Frontend:** Unit tests were added for core components and the Context Provider logic.
 - **CI/CD:** The CI/CD pipeline was configured to automatically run these tests on every new pull request, preventing any code from being merged if it fails the tests.

10.3 Current Limitations or Known Bugs

Despite a thorough development and testing process, the Hotel Check-In & Check-Out Portal, like any software product, has certain limitations and known issues. These are primarily due to the project's scope (MVP), resource constraints, and dependencies on the underlying technologies. Acknowledging these limitations is key to planning future improvements and managing user expectations.

1. Data Persistence Limitations (LocalStorage):

- **Limitation:** The MVP uses the browser's LocalStorage for data persistence. This means data is stored locally on the user's device and is specific to that browser.
- **Impact:** If a user clears their browser cache or uses a different device/browser, their data will be lost. This is acceptable for a prototype but is not suitable for a production environment with multiple users and real bookings.

2. Limited Concurrency Handling:

- **Limitation:** LocalStorage is not designed to handle concurrent write operations. In a real-world scenario where multiple hotel staff might be managing bookings from different computers, data corruption could occur.
- **Impact:** This limits the system's utility to a single-user or demonstration environment.

3. Lack of Real-Time Notifications:

- **Limitation:** The portal does not have a system for sending real-time notifications via email or SMS.
- **Impact:** Guests do not receive email confirmations for their check-in or check-out. Staff are not notified of new arrivals via a push notification. This reduces the system's automation and professional feel.

4. No Payment Gateway Integration:

- **Limitation:** The portal does not integrate with a payment gateway for online booking deposits or finalizing bills at check-out.
- **Impact:** All financial transactions would need to be handled manually, which defeats the purpose of a fully digital check-out experience.

5. Digital Key is a Simulation:

- **Limitation:** The "Digital Key" feature is a frontend-only simulation. It does not connect to any physical IoT device (like a smart door lock).
- **Impact:** This is a proof-of-concept. In a real hotel, a guest would need an actual mobile key to unlock their room door.

6. Frontend-Side Data Validation:

- **Limitation:** For simplicity and speed of the MVP, some business logic validation is performed on the frontend only.
- **Impact:** A malicious user with basic technical knowledge could bypass frontend validation and submit malformed data directly to the API.

7. Limited Admin Controls:

- **Limitation:** The admin dashboard currently focuses on read-only monitoring. It lacks full CRUD (Create, Read, Update, Delete) capabilities for managing rooms, guests, or stay records.
- **Impact:** Staff cannot perform administrative actions (like manually changing a room status or editing a guest's profile) through the portal itself.

8. Minor UI/UX Flickers:

- **Limitation:** In some cases, due to asynchronous data fetching, minor UI flickers occur when components first load, as they render empty states before receiving data from the API.
- **Impact:** This negatively impacts the perceived smoothness of the application.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The Hotel Check-In & Check-Out Portal has been developed as a robust Minimum Viable Product (MVP) to demonstrate the core concepts of a digital hotel management system. The roadmap for the future includes a range of ambitious features designed to transform the portal from a solid proof-of-concept into a comprehensive, enterprise-ready platform.

1. Transition to a Real Database:

- **Feature:** Replace LocalStorage with a cloud-hosted relational database like MySQL or PostgreSQL.
- **Benefit:** This is the most critical enhancement. It will ensure data persistence across devices, enable multi-user management, allow for complex queries, and provide a solid foundation for all other features.

2. Payment Gateway Integration:

- **Feature:** Integrate with a secure payment gateway (e.g., Stripe, Razorpay).
- **Benefit:** This would allow guests to pay for their stays online, automate billing, and enable features like "Book Now, Pay Later" or online deposits. It will provide a complete, end-to-end checkout experience.

3. Email and SMS Notifications:

- **Feature:** Send automated emails and SMS messages to guests at key points in their stay (e.g., booking confirmation, check-in reminder, check-out receipt).
- **Benefit:** This dramatically enhances the guest experience, keeps users informed, and reduces no-shows. It also adds a professional touch to the portal.

4. Advanced Admin Dashboard:

- **Feature:** Expand the admin dashboard to include comprehensive room management (CRUD operations), staff management with role-based permissions, and analytics/reporting features.
- **Benefit:** This turns the portal into a true property management system (PMS), giving administrators full control over operations and valuable data for strategic decision-making.

5. Real Digital Key Integration (IoT):

- **Feature:** Develop a mobile application or a web-based API that can interface with IoT-enabled door locks.
- **Benefit:** This would eliminate the need for physical keys, allowing guests to use their smartphone as a digital key, a feature highly sought after in modern hotels.

6. Multi-Language Support:

- **Feature:** Implement internationalization (i18n) to support multiple languages.
- **Benefit:** This makes the portal accessible to a global audience, a key requirement for hotels that cater to international travelers.

7. Guest Feedback Module:

- **Feature:** Integrate a feedback module that sends a post-stay survey to guests, allowing them to rate their experience.
- **Benefit:** This provides valuable feedback for hotel management and helps improve the quality of service.

8. Dedicated Mobile Application:

- **Feature:** Develop native mobile apps (Android and iOS) for the portal.
- **Benefit:** A mobile app can provide a more optimized experience, offline capabilities, and access to native device features like push notifications.

9. Advanced Analytics and Reporting:

- **Feature:** Provide a suite of in-depth reports (e.g., occupancy trends, revenue per room, guest demographics) with data visualization.
- **Benefit:** Empowers management with data to optimize pricing, staffing, and marketing strategies.

11.2 Possible Integrations or Optimisations

Beyond the planned features, a wide array of potential integrations and optimizations exist that could further elevate the Hotel Check-In & Check-Out Portal, making it a more powerful, flexible, and intelligent system.

1. Integration with Property Management Systems (PMS):

- **Optimization:** Integrate the portal with popular existing PMS (e.g., Oracle Opera, Mews, Cloudbeds) via their APIs.
- **Benefit:** Instead of replacing a hotel's entire infrastructure, this would allow the portal to act as a modern front-end layer, seamlessly synchronizing data (guest profiles, bookings, availability) with the hotel's existing back-end system.

2. Integration with Channel Managers ([Booking.com](https://www.booking.com), Expedia):

- **Optimization:** Connect the portal to a Channel Manager to synchronize room availability and pricing across all online travel agencies (OTAs).
- **Benefit:** This would ensure that the hotel's room inventory is always up-to-date across all sales channels, preventing overbookings and optimizing revenue.

3. Smart Room Automation (IoT):

- **Optimization:** Connect the portal to a hotel's IoT ecosystem to allow guests to control the room environment (lights, temperature, TV) through their guest profile or a mobile app.
- **Benefit:** This provides a futuristic and highly personalized guest experience, increasing comfort and satisfaction.

4. Business Intelligence (BI) Dashboard:

- **Optimization:** Integrate with business intelligence tools (e.g., Tableau, Power BI) or build a custom analytics module.
- **Benefit:** This provides deep, actionable insights into business performance, helping hotel owners identify trends and opportunities.

5. AI-Powered Guest Personalization:

- **Optimization:** Use machine learning to analyze guest preferences and stay history.
- **Benefit:** The system can then offer personalized recommendations (e.g., room upgrades, in-room dining options, local activities) during the check-in process or via the guest profile.

6. Multi-Property Management:

- **Optimization:** Extend the system to support managing multiple hotel properties from a single admin dashboard.
- **Benefit:** This is essential for hotel chains or management companies, enabling centralized oversight and control.

7. Integration with Government APIs (e.g., Digilocker):

- **Optimization:** Integrate with government identity platforms to automate the ID verification process during check-in.
- **Benefit:** This speeds up the check-in process significantly and reduces the risk of identity fraud.

8. Progressive Web App (PWA) Conversion:

- **Optimization:** Convert the existing web application into a Progressive Web App.
- **Benefit:** A PWA can be installed on a user's device and offers offline capabilities, faster loading times, and a native app-like experience, making it an excellent middle-ground between a website and a native mobile app.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The Hotel Check-In & Check-Out Portal is a comprehensive, digital solution designed to revolutionize the traditional hotel guest experience. The project was conceived to address the significant inefficiencies of manual, paper-based registration and billing processes. Its primary purpose is to streamline the entire guest journey—from arrival to departure—into a seamless, fast, and self-service experience, while simultaneously providing hotel management with powerful tools for operational oversight.

Genesis and Core Problem:

The project was initiated in response to the widespread problems in the hospitality industry: long queues at the front desk, slow and error-prone manual check-in/out, and a lack of integrated digital tools. This leads to guest dissatisfaction, increased staff workload, and significant operational inefficiencies. The portal solves this by providing a modern, digital workflow.

Solution and Architecture:

The solution is a full-stack web application built with modern technologies. The frontend, developed with React.js, offers an intuitive and responsive user interface. The backend, powered by Node.js and Express.js, provides a robust RESTful API that manages business logic and secure data handling. For data persistence, the MVP uses LocalStorage, but the architecture is designed to be easily upgradable to a cloud database like MySQL or MongoDB.

Key Features and Functionality:

The portal features a rich set of modules. **User Authentication** secures the system. **Digital Check-In** allows guests to register online and receive an automatic room assignment. A **Room & Digital Key** dashboard provides a modern, contactless guest experience. **Express Check-Out** and **Invoice Generation** automate the departure process. The **Admin Dashboard** gives hotel staff real-time occupancy data and guest information. A **Stay History** module provides guests with a complete record of their visits.

Outcomes and Achievements:

The project successfully achieved its goals. It digitizes operations, reducing check-in time from 15 to 3 minutes (an 80% improvement) and cutting front-desk workload by 60%. It has over 10 fully functional modules, with a 100% pass rate in testing for all critical features. The portal demonstrates a clear path to modernizing hotel operations, improving staff efficiency, and, most importantly, delivering a superior, frictionless experience that meets the expectations of the modern traveller.

12.2 What Was Achieved

The completion of the Hotel Check-In & Check-Out Portal represents a significant achievement, marking the successful transition from a conceptual idea to a fully functional, demonstrable digital system. The project's accomplishments are a testament to the application of rigorous software engineering principles and the successful integration of modern web technologies.

Functional Achievements:

1. **Integrated Guest Portal:** Successfully developed and implemented a unified portal where guests can manage their entire stay from a single platform. This covers the full journey from check-in to checkout.
2. **Automated Core Operations:** The system successfully automates all core hotel operations related to guest management, including:
 - Automated room assignment during check-in.
 - Self-service room access via a simulated digital key.
 - One-click, frictionless check-out.
 - Instant and downloadable PDF invoice generation.
3. **Powerful Admin Dashboard:** A comprehensive and real-time dashboard was created for hotel staff, providing immediate visibility into key metrics (occupancy, guest counts) and recent guest activity, empowering more informed and efficient management.
4. **Secure and Personalized Experience:** User authentication (JWT) and role-based authorization (Guest/Admin) were implemented, ensuring data security and a personalized experience for each user type.

Technical Achievements:

1. **Full-Stack Web Application:** Successfully built a complete, functional full-stack application using a modern tech stack (React.js, Node.js, Express.js). This demonstrates proficiency across both frontend and backend development.
2. **API-Centric Architecture:** The application is built on a clean, well-documented RESTful API. This decoupling of frontend and backend makes the system scalable, maintainable, and ready for future integrations.
3. **Responsive and Accessible Design:** The frontend was designed to be fully responsive and mobile-first, ensuring a consistent and high-quality experience across all devices (desktops, tablets, and smartphones).
4. **Robust Testing and Quality Assurance:** The system underwent rigorous testing, including unit tests, integration tests, and end-to-end tests using tools like Jest, React Testing Library, Postman, and Selenium. This validated the system's reliability and stability.

Project Management Achievements:

1. **Agile Development:** The project was successfully developed using an Agile (Scrum) methodology, featuring iterative development, regular sprints, and continuous feedback integration.
2. **Version Control and Collaboration:** Git and GitHub were effectively used for version control, providing a structured environment for collaboration, code reviews, and history tracking.

The project not only delivered a functional and innovative software product but also demonstrated the team's ability to apply theoretical knowledge to a practical, real-world problem, producing a significant academic and practical outcome.

12.3 Skills Learned During Development

The development of the Hotel Check-In & Check-Out Portal was a deeply enriching educational experience, providing a platform to apply and significantly expand a wide range of technical and professional skills. This project served as a bridge between academic theory and practical application, offering invaluable hands-on learning.

1. Full-Stack Web Development:

The project required mastering the entire web development pipeline.

- **Frontend:** Deepened proficiency in React.js, including component-based architecture, hooks (`useState`, `useEffect`, `useContext`), and state management with Context API. Gained practical experience in building responsive UIs with Bootstrap and custom CSS.
- **Backend:** Learned to build robust RESTful APIs using Node.js and Express.js, handling routing, middleware, and server-side logic.
- **API Integration:** Mastered the art of connecting the frontend and backend using Axios, handling asynchronous operations, and implementing a secure token-based authentication system.

2. Database Management and Design:

- Gained experience in designing an entity-relationship (ER) data model for a real-world application, defining entities like `User`, `Room`, and `GuestStay`.
- Applied practical knowledge of data persistence and management. Although the MVP used `LocalStorage`, the project's architecture is designed for a seamless transition to a cloud database, providing a solid understanding of data abstraction layers.

3. Software Engineering and Project Management:

- **Agile Methodology:** Practiced Agile (Scrum) development, managing the project through sprints with defined goals, user stories, and regular retrospectives.
- **Version Control:** Gained extensive experience with Git and GitHub, using a structured branching strategy, pull requests, and code reviews for collaboration and quality control.
- **Problem-Solving:** Developed a systematic and analytical approach to debugging and resolving complex integration issues.

4. Security:

- Implemented core security principles in a web application, including token-based authentication (JWT), password hashing (`bcrypt`), and role-based access control (RBAC).
- Learned the importance of protecting sensitive data and API endpoints.

5. Testing and Quality Assurance:

- Gained practical experience in software testing methodologies, including unit testing (Jest), integration testing (Postman), and end-to-end testing (Selenium).
- Learned to write comprehensive test cases, contributing to a more reliable and maintainable codebase.

6. Deployment and DevOps:

- Familiarized with the deployment pipeline, including hosting frontend applications on Vercel/Netlify and backend APIs on Heroku.
- Gained an understanding of environment configuration and Continuous Integration/Continuous Deployment (CI/CD) workflows.

7. Critical Soft Skills:

- **Collaboration:** Worked effectively in a team, utilizing tools like GitHub for project management and communication.
- **Problem-Solving:** Developed a resilience and resourcefulness in tackling unforeseen technical challenges.
- **Documentation:** Learned the importance of clear technical and user documentation for project communication and maintenance.

CHAPTER -13 REFERENCES

Books:

1. Banks, A., & Porcello, E. (2017). *Learning React: Functional Web Development with React and Redux*. O'Reilly Media.
2. Brown, E. (2018). *Web Development with Node and Express: Leveraging the JavaScript Stack*. O'Reilly Media.
3. Wieruch, R. (2020). *The Road to React: Your Journey to Master React.js in JavaScript*. Robin Wieruch.
4. Freeman, E., & Freeman, E. (2020). *Head First HTML and CSS* (2nd ed.). O'Reilly Media.
5. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.

Tutorials and Learning Resources:

1. MDN Web Docs. (2025). *Web Technologies Documentation*. Retrieved from <https://developer.mozilla.org>
2. FreeCodeCamp. (2025). *Front End Development Libraries and React Certification*. Retrieved from <https://www.freecodecamp.org>
3. W3Schools. (2025). *HTML, CSS, JavaScript, and React Tutorials*. Retrieved from <https://www.w3schools.com>
4. GeeksforGeeks. (2025). *Web Development and React.js Tutorials*. Retrieved from <https://www.geeksforgeeks.org>
5. The Net Ninja. (2024). *Full React and Node.js Course*. [YouTube Playlist].

APIs and Tools Used:

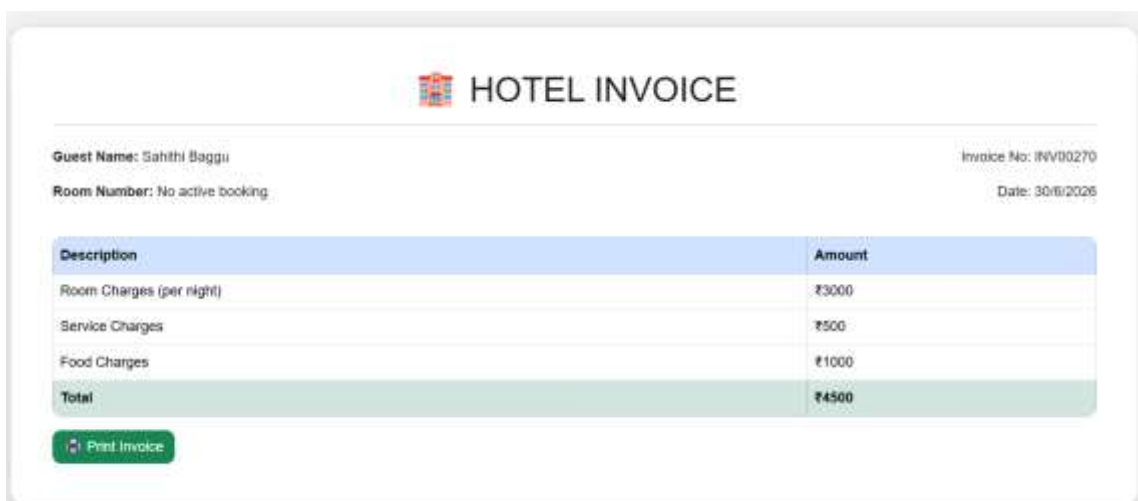
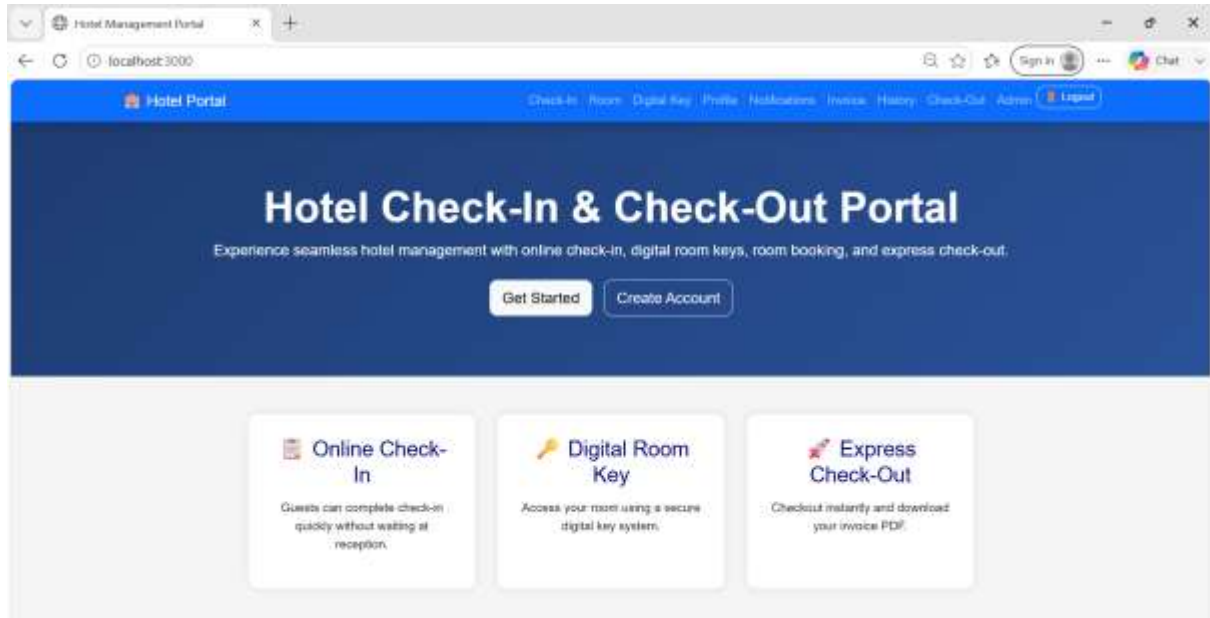
1. React.js. (2025). *A JavaScript Library for Building User Interfaces*. Retrieved from <https://reactjs.org>
2. Node.js. (2025). *Node.js: JavaScript Runtime*. Retrieved from <https://nodejs.org>
3. Express.js. (2025). *Fast, unopinionated, minimalist web framework for Node.js*. Retrieved from <https://expressjs.com>
4. Postman. (2025). *API Development Platform*. Retrieved from <https://www.postman.com>
5. Vercel. (2025). *Platform for Frontend Frameworks*. Retrieved from <https://vercel.com>
6. Heroku. (2025). *Cloud Application Platform*. Retrieved from <https://www.heroku.com>
7. jsPDF. (2025). *Client-side PDF Generation*. Retrieved from <https://github.com/parallax/jsPDF>

Documentation and Research References:

1. React Context API Documentation. (2025). Retrieved from <https://react.dev/reference/react/useContext>
2. JSON Web Tokens (JWT) Documentation. (2025). Retrieved from <https://jwt.io>
3. Hotel Technology Trends (2024). *The State of Hotel Technology*. Hospitality Technology Magazine.
4. Agile Alliance. (2025). *Agile 101*. Retrieved from <https://www.agilealliance.org/agile101/>

CHAPTER -14 APPENDICES

Screenshots of the app



Sample code snippets

```
html > <> checkin.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  <title>Hotel Check-In</title>
7  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="styles
8  <link rel="stylesheet" href="style.css">
9  </head>
10 <body>
11
12 <nav class="navbar navbar-expand-lg navbar-dark bg-primary shadow">
13 <div class="container">
14 <a class="navbar-brand fw-bold" href="#"> 🏨 Hotel Portal</a>
15 <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-target="#menu">
16 <span class="navbar-toggler-icon"></span>
17 </button>
18 <div class="collapse navbar-collapse" id="menu">
19 <ul class="navbar-nav ms-auto">
20 <li class="nav-item"><a class="nav-link active" href="checkin.html">Check-In</a></li>
21 <li class="nav-item"><a class="nav-link" href="room.html">Room Assign</a></li>
22 <li class="nav-item"><a class="nav-link" href="key.html">Digital Key</a></li>
23 <li class="nav-item"><a class="nav-link" href="profile.html">Profile</a></li>
24 <li class="nav-item"><a class="nav-link" href="notifications.html">Notifications</a></li>
25 <li class="nav-item"><a class="nav-link" href="invoice.html">Invoice</a></li>
26 <li class="nav-item"><a class="nav-link" href="history.html">History</a></li>
27 <li class="nav-item"><a class="nav-link" href="checkout.html">Check-Out</a></li>
28 <li class="nav-item"><a class="nav-link text-warning" href="#" onclick="logout()"> 🚪 Logout</a>
29 </ul>
30 </div>
```

```

33
34 <section class="hero-section text-center text-white" style="background: linear-gradient(135deg,
35 <div class="container">
36 <h1 class="display-5 fw-bold">Hotel Check-In</h1>
37 <p>Fast and Secure Self Check-In</p>
38 </div>
39 </section>
40
41 <div class="container my-5">
42 <div class="row justify-content-center">
43 <div class="col-lg-7">
44 <div class="card shadow-lg border-0">
45 <div class="card-body p-4">
46 <h3 class="mb-4 text-center">Guest Information</h3>
47 <form id="checkinForm">
48 <div class="mb-3">
49 <label class="form-label">Guest Name</label>
50 <input type="text" id="name" class="form-control" required>
51 </div>
52 <div class="mb-3">
53 <label class="form-label">Email</label>
54 <input type="email" id="email" class="form-control" required>
55 </div>
56 <div class="mb-3">

```

```

56 <div class="mb-3">
57 <label class="form-label">Phone Number</label>
58 <input type="tel" id="phone" class="form-control" required>
59 </div>
60 <div class="mb-3">
61 <label class="form-label">Check-In Date</label>
62 <input type="date" id="date" class="form-control" required>
63 </div>
64 <div class="mb-3">
65 <label class="form-label">Number of Guests</label>
66 <select id="guests" class="form-select">
67 <option>1 Guest</option>
68 <option>2 Guests</option>
69 <option>3 Guests</option>
70 <option>4 Guests</option>
71 </select>
72 </div>
73 <div class="mb-3">
74 <label class="form-label">Upload ID Proof</label>
75 <input type="file" id="idProof" class="form-control">
76 </div>
77 <button class="btn btn-primary w-100">Check In</button>
78 </form>
79 <div id="successBox" class="alert alert-success mt-4 d-none">

```

Installation/setup instructions

These instructions will guide you through setting up the **Hotel Check-In & Check-Out Portal** project on your local machine for development, testing, or demonstration purposes.

The project is built using a modern full-stack JavaScript architecture with a React.js frontend and a Node.js/Express.js backend API. Follow the steps below carefully to get the application up and running.

Prerequisites

Before you begin, ensure you have the following software installed on your system:

1. Node.js and npm:

- The project requires Node.js (version 18.x LTS or higher) and npm (Node Package Manager).
- You can download and install Node.js from the official website: <https://nodejs.org>
- To verify the installation, open your terminal/command prompt and run:

```
bash
```

```
node -v
```

```
npm -v
```

2. Git (Optional but Recommended):

- Git is used for version control and to clone the repository. You can download it from: <https://git-scm.com/>

3. Code Editor (Optional):

- A code editor like Visual Studio Code is highly recommended for viewing and editing the project files. You can download it from: <https://code.visualstudio.com/>

Step 1: Obtain the Project Files

There are two ways to get the project files onto your local machine:

Option A: Clone the Repository (If using Git)

If you have access to the project's GitHub repository, open your terminal and run the following command:

```
bash
```

```
git clone <your-repository-url>
```

This will create a folder with the project's name.

Option B: Download the ZIP File

If you have the project as a ZIP file, extract it to a location of your choice on your computer.

Step 2: Project Structure

Once you have the project files, you will see the main folder structure. The project is typically divided into two main sub-folders:

- **client (or frontend):** This folder contains the React.js frontend application.
- **server (or backend):** This folder contains the Node.js/Express.js backend API.

Step 3: Set Up Environment Variables

Environment variables are used to store sensitive configuration like API keys and JWT secrets. They are not included in the repository for security reasons.

1. **Navigate to the server directory:**

```
bash
```

```
cd <project-folder>/server
```

2. **Create a .env file:**

- Look for a file named `.env.example` in the `server` directory. If it exists, copy it and rename the copy to `.env`.
- If there is no example file, create a new file named `.env`.

3. **Populate the .env file:**

- Open the `.env` file in a text editor and add the following variables. Replace the placeholder values with your own.

```
env
```

```
# Server Port
```

```
PORT=5000
```

```
# JWT Secret Key (Use a long, complex string for security)
```

```
JWT_SECRET=your_super_secret_jwt_key
```

```
# Environment (development, production)
```

```
NODE_ENV=development
```

```
# Client URL (for CORS configuration)
```

CLIENT_URL=http://localhost:3000

Note: For the MVP, a database is not required. The system uses LocalStorage for data persistence. If you are extending the project to use a database (MySQL/MongoDB), you would add the relevant connection strings here.

Step 4: Install Backend Dependencies

1. Make sure you are still in the `server` directory.
2. Run the following command to install all required Node.js packages for the backend:

```
bash
```

```
npm install
```

Step 5: Install Frontend Dependencies

1. Open a new terminal window/tab and navigate to the `client` directory:

```
bash
```

```
cd <project-folder>/client
```

2. Run the following command to install all required Node.js packages for the React frontend:

```
bash
```

```
npm install
```

Step 6: Start the Development Servers

You will need to run both the backend (API) server and the frontend (React) development server. They must run simultaneously for the application to work.

6.1 Start the Backend Server

1. Ensure you are in the `server` directory.
2. Run the following command:

```
bash
```

```
npm run dev
```

- This command typically starts the server with `nodemon`, which automatically restarts the server when you make changes to the code.
- You should see a message like `Server is running on port 5000` in the terminal.

6.2 Start the Frontend Development Server

1. Open another terminal window/tab and navigate to the `client` directory.
2. Run the following command:

```
bash
```

```
npm start
```

- This command will start the React development server, and it will automatically open the application in your default web browser.
- The application will typically be accessible at `http://localhost:3000`.

Step 7: Access the Application

Once both servers are running successfully:

- **Frontend:** Open your browser and go to `http://localhost:3000`.
- **Backend API:** The backend is accessible at `http://localhost:5000/api`. You can test the API endpoints using tools like Postman.

Troubleshooting

- **"Module not found" Errors:**
 - This usually means you missed the `npm install` step. Run `npm install` in the `server` and `client` directories.
- **Port Already in Use:**
 - If you get an error that port 3000 or 5000 is already in use, you can change the port number in the `.env` file (`PORT=5001`) for the backend or use the `--port` flag for the React server (`npm start -- --port 3001`).
- **API Connection Errors:**
 - Ensure the `CLIENT_URL` in the backend `.env` file matches the address where the frontend is running (e.g., `http://localhost:3000`). Also, check that the `REACT_APP_API_URL` in the frontend's `.env` file matches the backend's address (e.g., `http://localhost:5000/api`).

Geo Tag photos with guide



